

TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP TP.HCM

Khoa Công Nghệ Điện Tử - Bộ môn Điện Tử Máy Tính

BÀI GIẢNG

**HỆ THỐNG NHÚNG
(Embedded System)**

TP.HCM-2024

NỘI DUNG MÔN HỌC

Chương 1: GIỚI THIỆU HỆ THỐNG NHÚNG

Chương 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

Chương 3: CẤU TRÚC PHẦN MỀM

Chương 4: THIẾT KẾ HỆ THỐNG NHÚNG

CHƯƠNG 2: KỸ THUẬT GIAO TIẾP NGOẠI VI



1. GIỚI THIỆU
2. CÁC LOẠI PHƯƠNG THỨC GIAO TIẾP
3. **MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN (TRÊN HTN)**

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

Contents

☐ UART

☐ I2C

☐ CAN

☐ SPI

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

☐ UART

☐ I2C

☐ CAN

☐ SPI

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

3.1.1 Introduction

- + **UART** or **Universal Asynchronous Receiver Transmitter** is a dedicated hardware associated with serial communication.
- + The hardware for **UART** can be a circuit integrated on the microcontroller or a dedicated IC.
- + **UART** is a serial communication device that performs parallel-to-serial data conversion at the **transmitter** side and serial-to-parallel data conversion at the **receiver** side.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

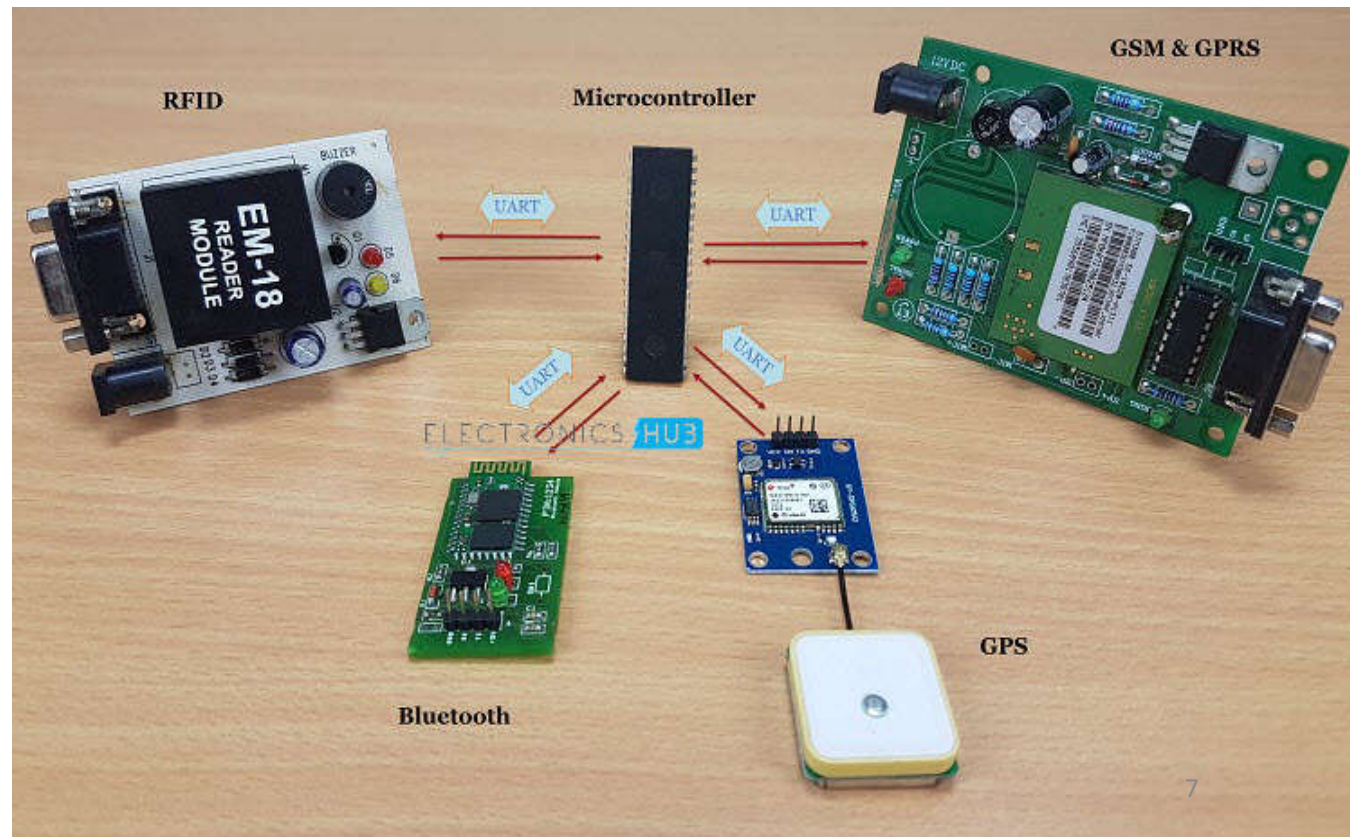
CHƯƠNG 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

3.1 UART

3.1.1 Introduction

+ **UART** is being used in many applications like:

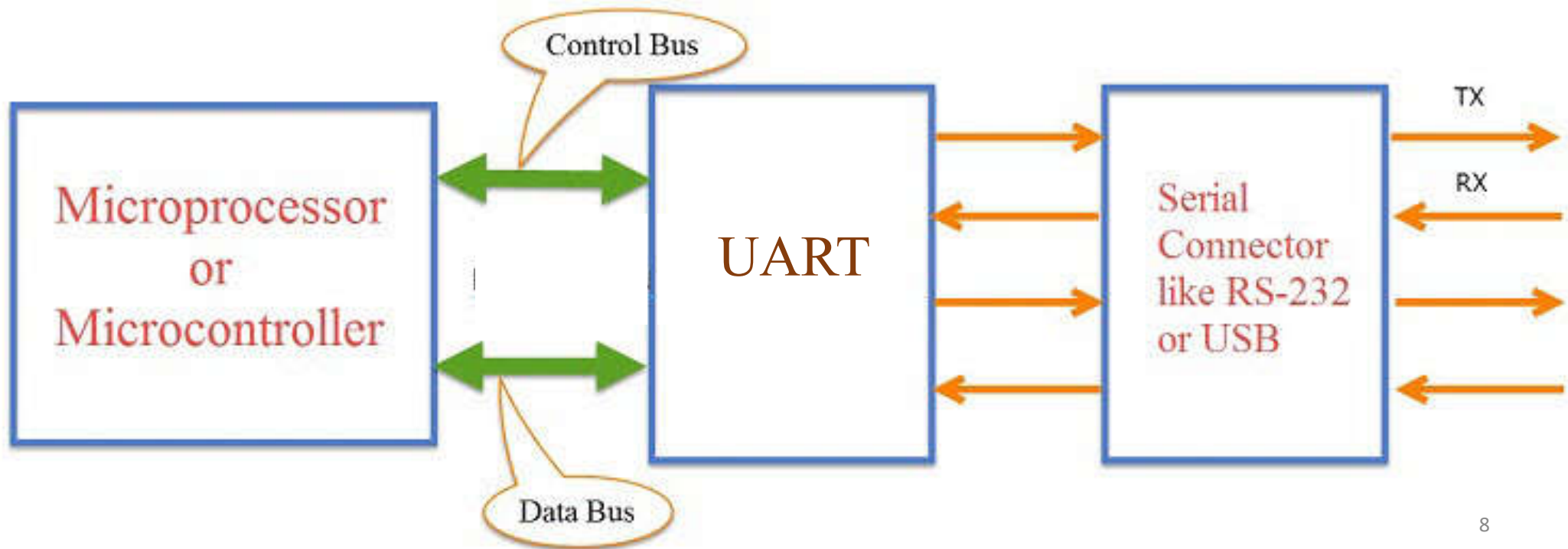
- **GPS Receivers,**
- **Bluetooth Modules,**
- **GSM-GPRS Modems,**
- **Wireless Communication Systems,**
- **RFID** based applications.



3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

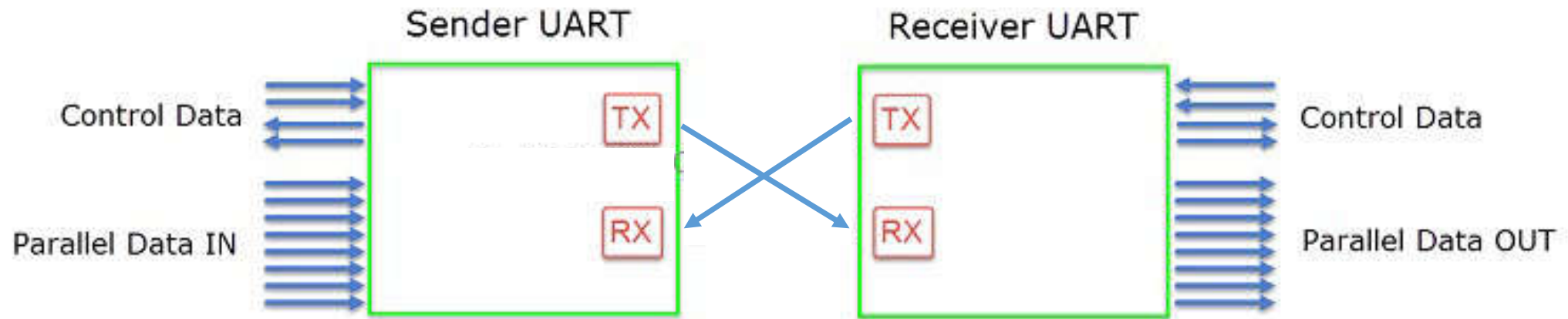
+ **UART** acts as a bridge between the processor and the serial communication protocol or port.



3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

- + The transmitting UART receives parallel data from the CPU (microprocessor / microcontroller) and converts it in to serial data (TX pin).
- + This serial data is transmitted to the receiving UART (by RX pin), which converts it back to parallel data and gives it to the CPU.



3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

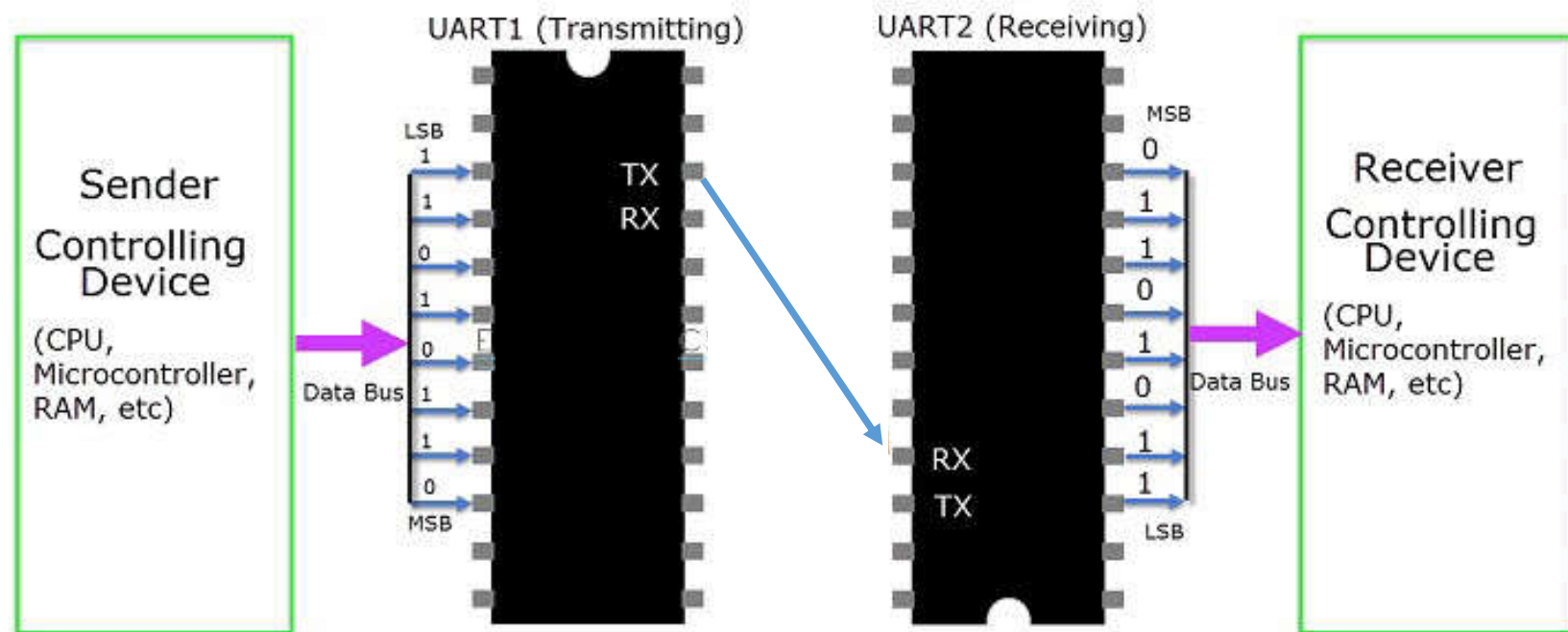
3.1.2 The operation of UART

- + The **transmitting UART** receives data from the controlling device through the data bus. The data received by the **transmitting UART** from the data bus is parallel data.
- + Then, UART adds a **start bit**, a **parity bit**, and a **stop bit**, creating the data packet. Next, the data packet is output serially, bit by bit (at the Tx pin).
- + The **receiving UART** reads the data packet (Rx pin) and converts the data back into parallel form and removes the start bit, parity bit, and stop bits.
- + Finally, the **receiving UART** transfers the data packet in parallel to the data bus on the receiving end.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

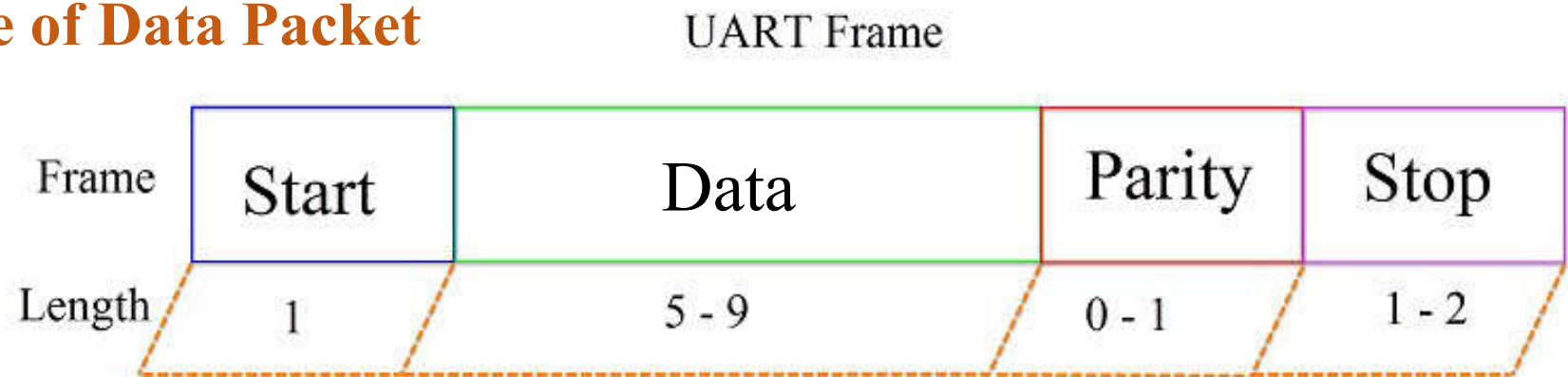
3.1.2 The operation of UART



3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

3.1.3 Structure of Data Packet

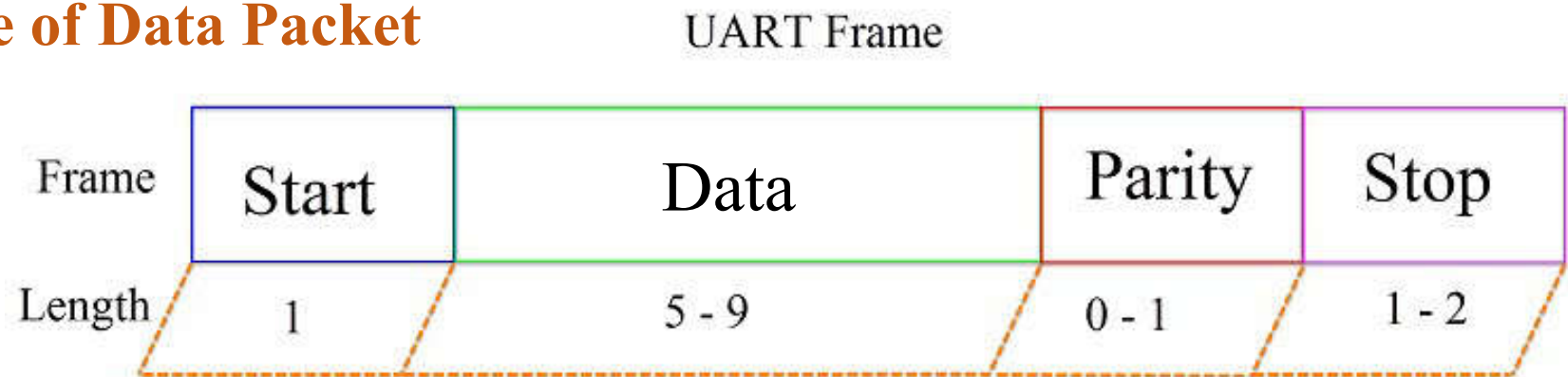


- + **Start bit**: is a synchronisation bit that is added before the actual data. Start bit marks the beginning of the data packet (1 → 0: to start the data transfer).
- + The **data frame** contains the actual data being transferred. It can be 5 bits up to 8 bits long if a parity bit is used. If no parity bit is used, the **data frame** can be 9 bits long. Usually, the LSB is the first bit of data to be transmitted.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

3.1.3 Structure of Data Packet



+ **Parity bit**: allows the receiver to check whether the received data is correct or not. Parity is **a low level error checking system** and comes in two varieties: Even Parity and Odd Parity.

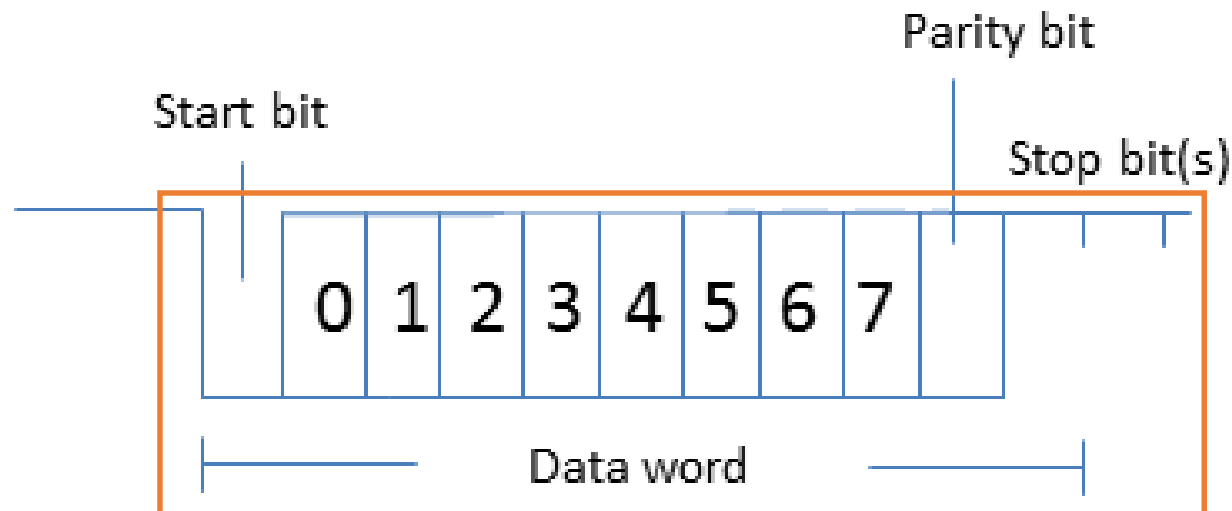
→ Parity bit is optional and it is actually not that widely used.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

3.1.3 Structure of Data Packet

+ **Stop bit**: marks the end of the data packet. It is usually two bits long but often only one bit is used. In order to end the transmission, the UART maintains the data line at **high voltage**.



3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

3.1.4 Steps of UART

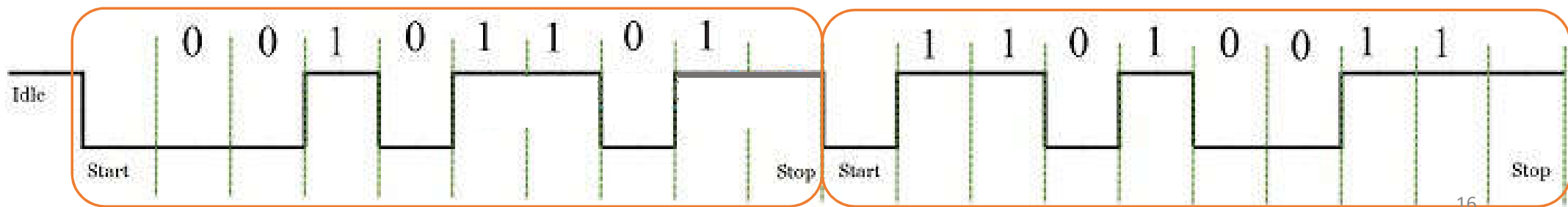
- + **Step_1**: The transmitting UART **receives** data in parallel from the data bus.
- + **Step_2**: The transmitting UART **adds** the **start** bit, **parity** bit, and the **stop** bit(s) to the **data frame**.
- + **Step_3**: The entire packet is sent serially from the transmitting UART to the receiving UART (*with the pre-configured **baud rate***).
- + **Step_4**: The receiving UART **discards** the start bit, parity bit, and stop bit from the data frame.
- + **Step_5**: The receiving UART **converts** the serial data back into parallel and transfers it to the data bus on the receiving end.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

3.1.4 Steps of UART

- + **Baud Rate**: The speed at which the data is transmitted. Both the transmitting UART and Receiving UART must agree on the same Baud Rate.
- + Baud Rate is measured in **bits per second**.
- + Some of the standard baud rates are 4800 bps, 9600 bps, 19200 bps, 115200 bps etc. (*An example, 9600 bps with 8 bits of data, no parity and 1 stop bit*)



3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN



CHƯƠNG 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

3.1 UART

3.1.5 USART (Universal Synchronous/Asynchronous Receiver/Transmitter)

- + A **USART** is hardware that enables a device to communicate using serial protocols.
- + **USART** can function in a *slower asynchronous mode*, like **UART**, or in a *faster synchronous mode* with a clock signal.
- + In this *synchronous mode* the transmitting peripheral will *generate a clock* that the receiving peripheral can recover from the data stream *without knowing* the baud rate ahead of time.
- + The use of the *external clock* allows the **data rate** of the USART to be much higher than that of a *standard UART*, reaching up to rates of **4 Mbps**.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

Synchronous and Asynchronous

	PROS	CONS
Synchronous communicate & program execution	• Enables Real-time processes. • Usually easier to track errors.	• Can be time-consuming or lead to latency while waiting for processes to complete sequentially. • User experience may be poor.
Asynchronous communicate & program execution	• Actions are not blocked by other requests are waiting to resolve. • Multiple actions can take place at one time, which decreases waiting time for users.	• Response times are not always predictable, which can effect time-sensitive processes. • Errors may be harder to track.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN



CHƯƠNG 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

3.1 UART

3.1.6 RS232 Protocol

What is RS232 ?

- + RS232 is a **Serial Communication Standard** developed by the **Electronic Industry Association (EIA)** and **Telecommunications Industry Association (TIA)**.
- + RS232 is one of the oldest serial communication standards and was introduced in **1960's** and was originally known as **EIA Recommended Standard 232** (is called **EIA/TIA-232**).
- + **UARTs** are frequently used in conjunction with the RS232 standard.

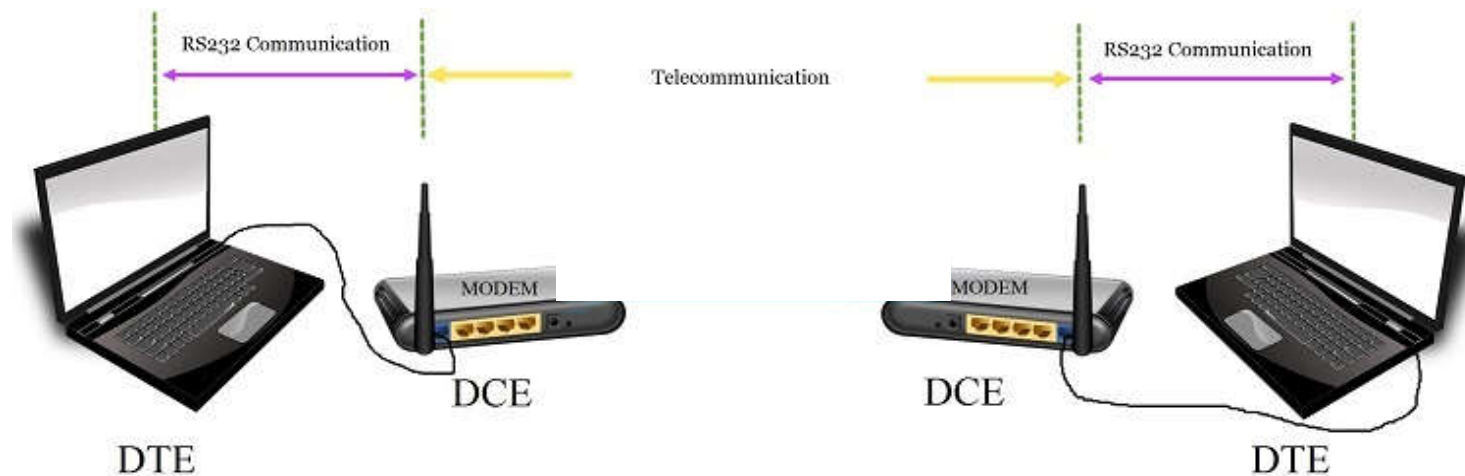
3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

3.1.6 RS232 Protocol

+ RS232 defines the signals connecting between DTE (Data Terminal Equipment) and DCE (Data Communication Equipment).

→ *A RS232 application between a Computer (DTE) and a Modem (DCE)*



3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

3.1.6 RS232 Protocol

+ RS232 ensures that there is no conflict between the DTE and DCE devices by specifying:

- Electrical specification
- Mechanical specification
- Functional specification
- Procedural Specifications

+ All these specifications provide user different parameters: *common voltage levels*, *signal levels*, *pin wiring specifications*, *control data* between the host device and its peripheral etc.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

3.1.6 RS232 Protocol

❑ Electrical Characteristics

- *Voltage Levels: (in Table)*

Voltage Level		
	Historically	Typically
High	-15V to -3V	-12V
Low	+3V to +15V	+12V

- *Slew rate*: is the rate of change of signal levels, → limited to 30V/μs.
- *Line Impedance*: The impedance of the wire between the DTE and DCE devices is specified to be around 3Ω to 7Ω.

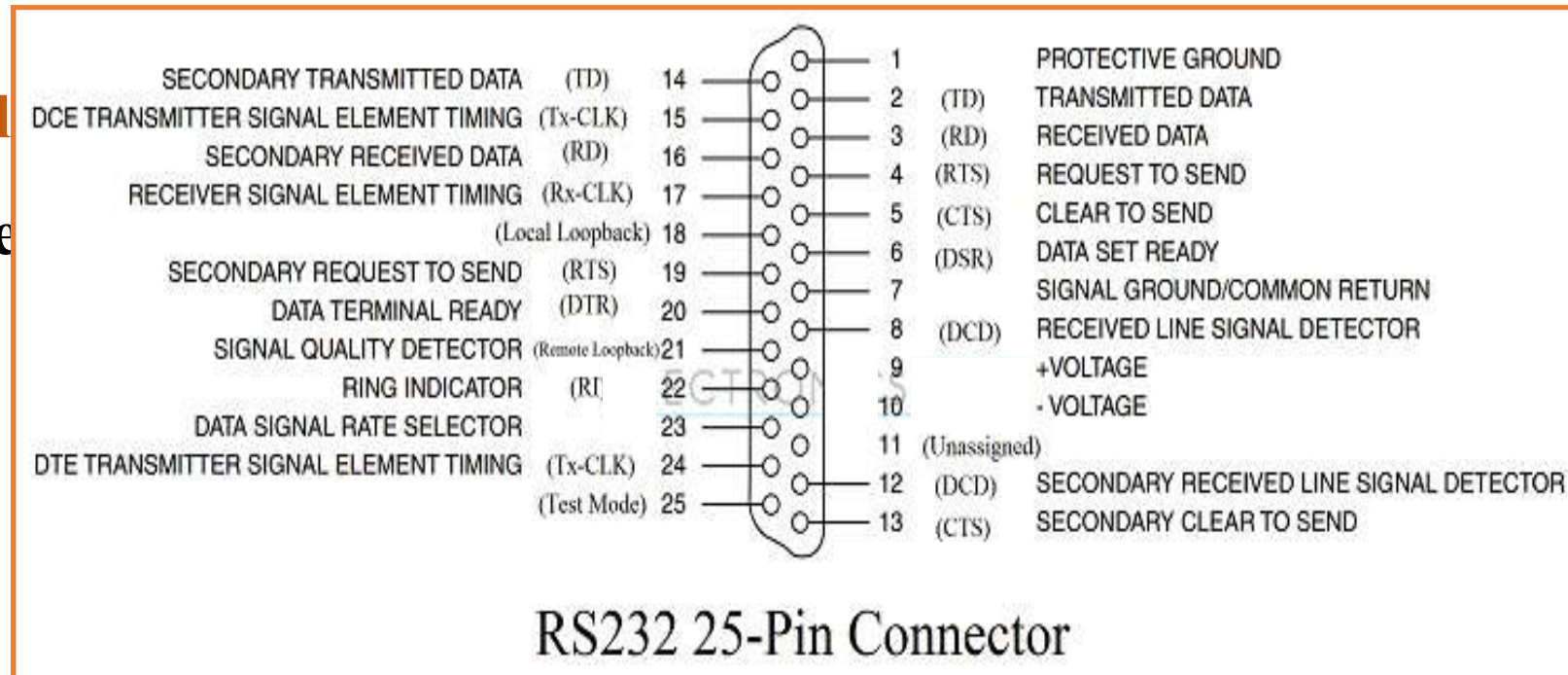
3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

CHƯƠNG 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

3.1 UART

3.1.6 RS232 Protocol

□ Mechanical Specifications



+ The Mechanical Specifications of RS232 includes the standard mechanical interface which specifies a **25-pin/D-Type Connector (DB-25)** to support the full functionality of RS232.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

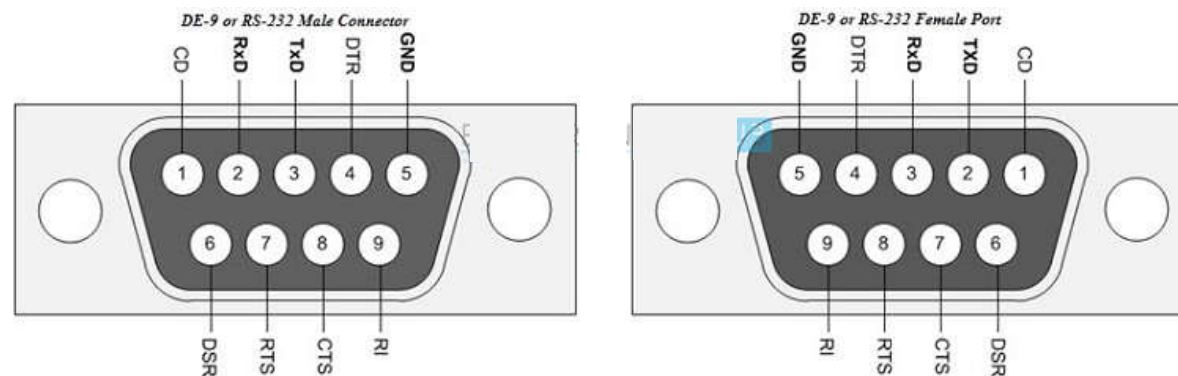
3.1 UART

3.1.6 RS232 Protocol

❑ Mechanical Specifications

+ As the electronic equipment and devices are getting **smaller**, and most common applications **do not require** all the 25-pin connector.

→ A reduced function **9-pin/D-Type connector (DE-9)** is used commonly.



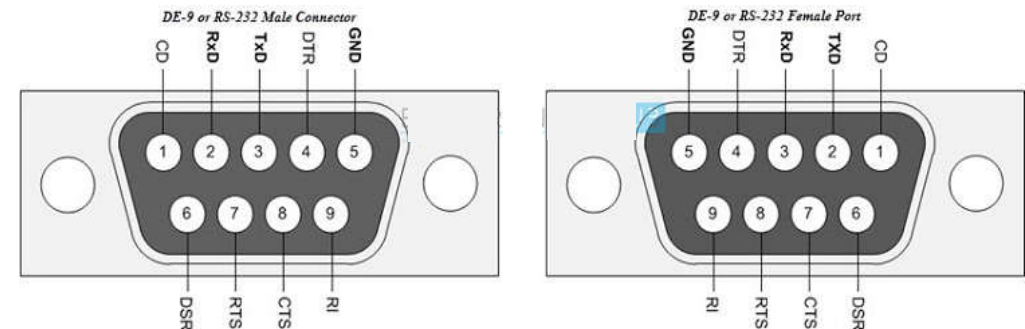
3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

CHƯƠNG 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

3.1 UART

3.1.6 RS232 Protocol

□ Mechanical Specifications



+ The pins in **DE-9 connector**, their names and description are given in the following Table.

Pin Number	Name	Direction
1	CD (Carrier Detect)	Incoming signal from DCE
2	RD (Receive Data)	Incoming Data from DCE
3	TD (Transmit Data)	Outgoing Data to DCE
4	DTR (Data Terminal Ready)	Outgoing handshaking signal
5	GND (Signal Ground)	Common reference voltage
6	DSR (Data Set Ready)	Incoming handshaking signal
7	RTS (Request to Send)	Outgoing flow control signal
8	CTS (Clear to Send)	Incoming flow control signal
9	RI (Ring Indicator)	Incoming signal from DCE

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

3.1.6 RS232 Protocol

□ Functional Specifications

- + Besides defining more than electrical and mechanical characteristics, RS232 standard also defines the functions of **different signals** used in the interface.
- + The signals are classified as: *Common; Data; Timing and Control Signals*.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

3.1.6 RS232 Protocol

- ❑ **Procedural Specifications** → specify the sequence of operations that must be carried out when a DTE and DCE are connected, as following.
 - When Modem (DCE) is **ready to receive**, it will **send a** (**DTR**) **ready signal**.
 - When the Computer (DTE) is **ready to send** the data, it **sends** a **Ready to Send (RTS) signal**.
 - The Modem (DCE) then **sends** a **Clear to Send (CTS) signal** to indicate that data can be sent by computer (DTE).
 - Finally, the Computer (DTE) **sends** data on **Transmit Data (TD) line** to the Modem (DCE).

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN



CHƯƠNG 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

3.1 UART

3.1.6 RS232 Protocol

Practical Implementation of RS232

+ The **voltage levels** of RS232 are **very different** from most of the systems designed today (TTL or CMOS logic).

→ **Need a level converter** of some sorts to implement RS232 interface (*by ICs like MAX232 by Maxim Integrated*).

+ **These ICs** take in the RS232 signals and generate a TTL level voltages.

→ **Invert the signals** as **low voltage** levels in RS232 are logic '1' and **high voltage** levels in RS232 are logic '0'.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

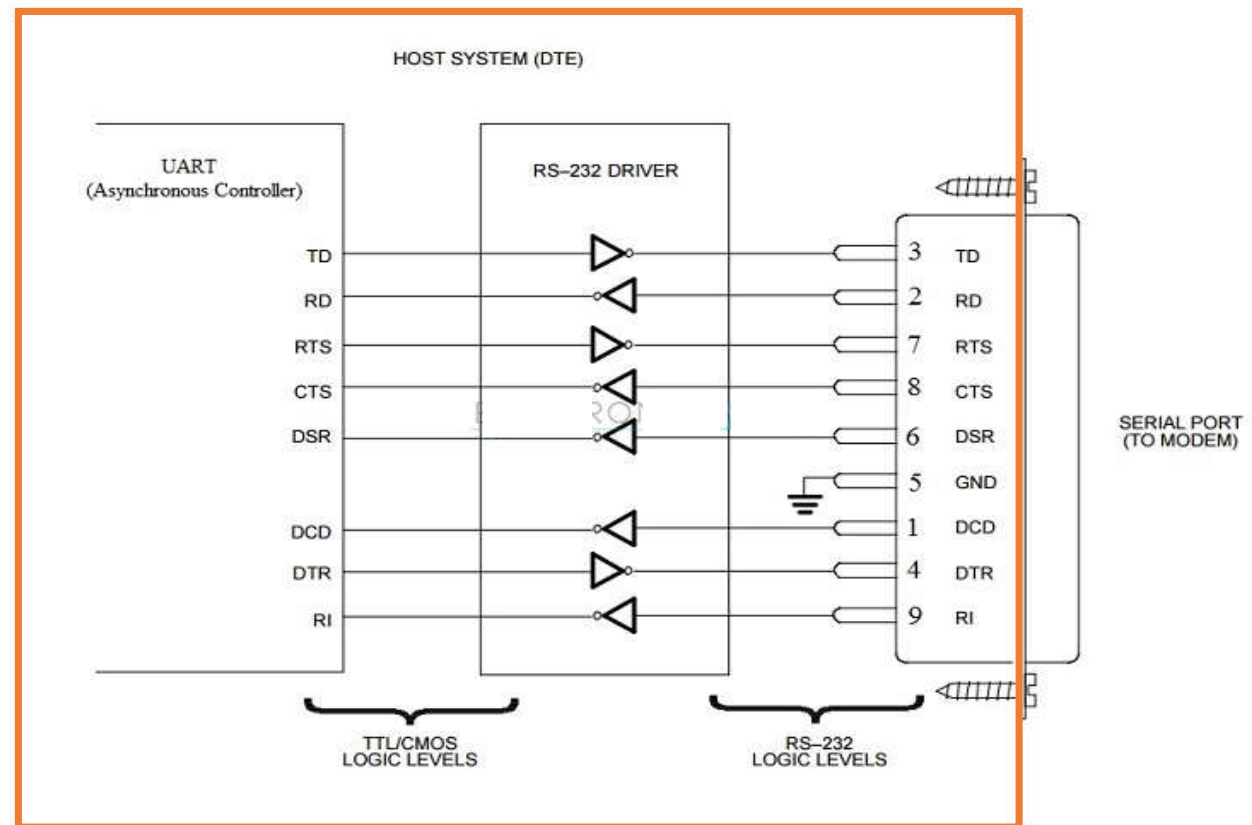
CHƯƠNG 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

3.1 UART

3.1.6 RS232 Protocol

Low voltage levels in RS232:
logic '1'.

High voltage levels in RS232:
logic '0'.



3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN



CHƯƠNG 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

3.1 UART

3.1.6 RS232 Protocol

Operation of RS232

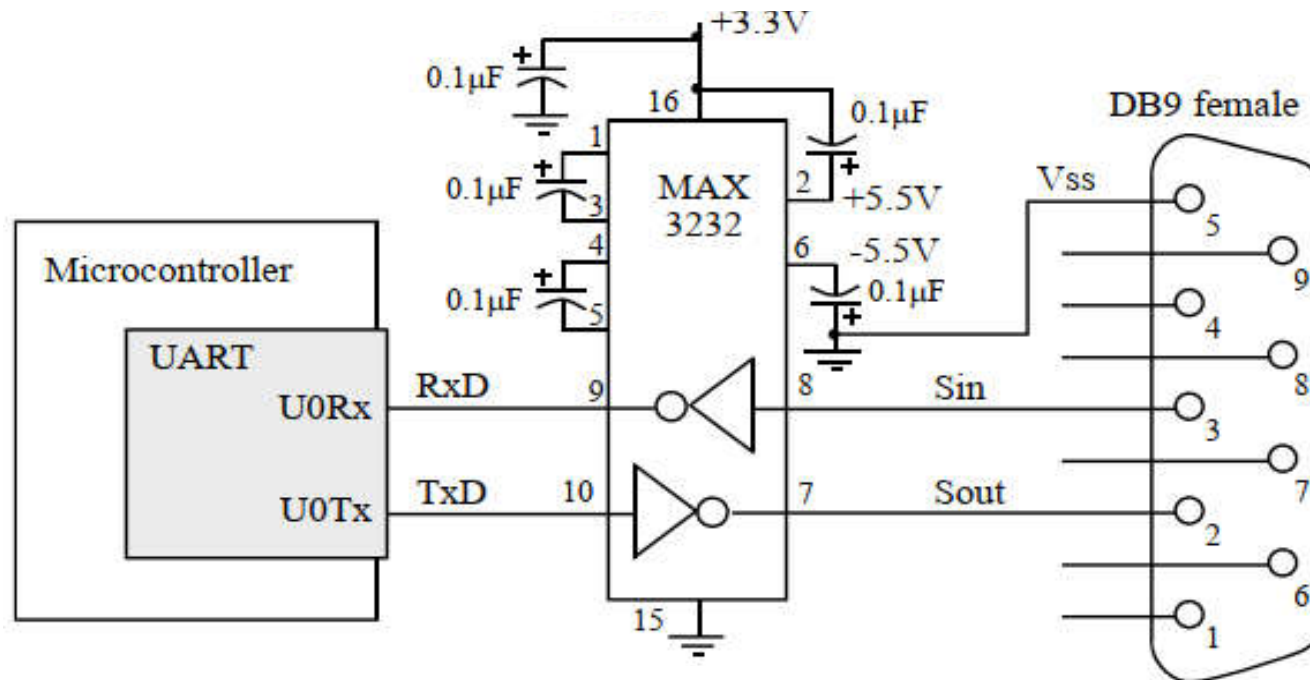
- + In RS232, the data is **transmitted serially** in one direction over a **single data line**.
- + In order to establish **two way communication**, we need at least three wires (RX, TX and GND) apart from the control signals.
- + RS232 follows **asynchronous communication** protocol i.e. there is **no clock signal** to synchronize transmitter and receiver.
 - It uses **start** and **stop bits** to inform the receiver when to check for data.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

3.1.6 RS232 Protocol

Connecting Schematic of MAX232



3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

3.1.6 RS232 Protocol

Handshaking

- + Handshaking is a process of dynamically *setting the parameters* of a communication between the transmitter and receiver before the communication begins.
- + The need for handshaking is dictated by *the speed at which the transmitter* (DTE) transmits the data, *the speed at which the receiver* (DCE) receives the data and *the rate at which the data is transmitted*.
- + In an **asynchronous** data transmission system, there can be **no Handshaking** (*hardware Handshaking* and *software Handshaking*).

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

3.1.6 RS232 Protocol

Handshaking

❑ No Handshaking

- + If handshaking is not used, then the receiver (DCE) must read the data that **is already received before** the transmitter (DTE) sends the next data.
- the receiver uses a special memory location called **Buffer** and since it is used at receiver end, it is called **Receiver Buffer**.
- + The Receiver Buffer can typically store **a single bit of data** and this data **must be cleared (read) before the next data arrives** → and if it is **not cleared**, the existing data will be **overwritten** by the new data.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

3.1.6 RS232 Protocol

Handshaking

□ Handshaking

- + In Hardware Handshaking, the transmitter **first asks** the receiver whether it is ready to receive the data. The receiver then **checks its Buffer** and if the Buffer is empty, it will then **tell the transmitter** that it is ready to receive.
- + The transmitter will transmit the data and it **is loaded into** the receiver Buffer. During this time, the receiver **tells** the transmitter not to send any further data **until** the **data in the Buffer has been read** by the receiver.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

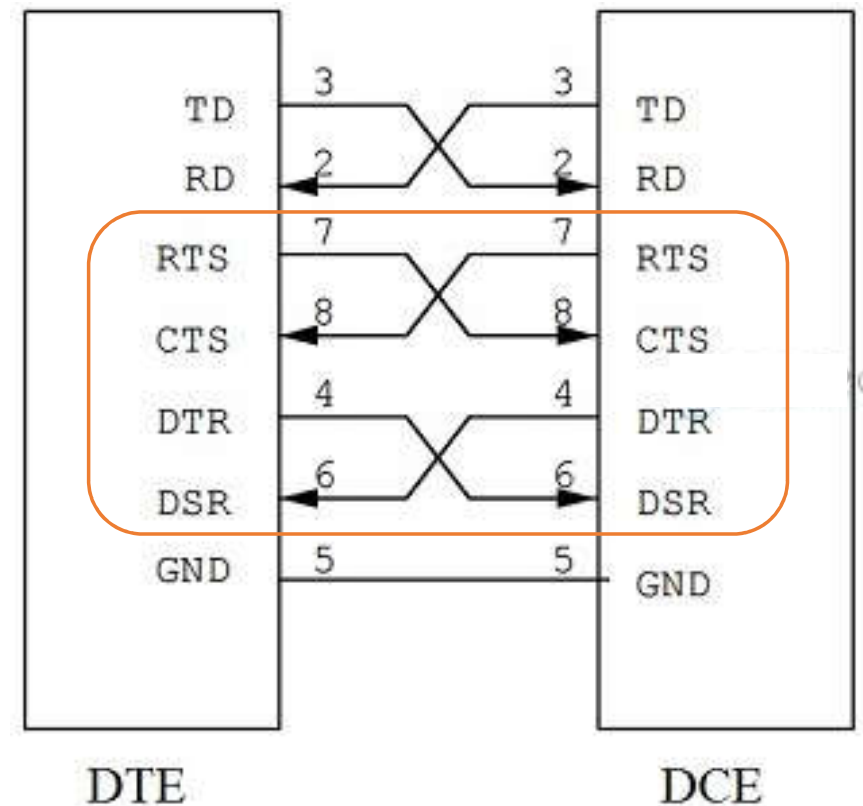
3.1 UART

3.1.6 RS232 Protocol

❑ Hardware Handshaking

+ The RS232 Protocol defines **four signals** for the purpose of Handshaking:

- Ready to Send (RTS)
- Clear to Send (CTS)
- Data Terminal Ready (DTR)
- Data Set Ready (DSR)



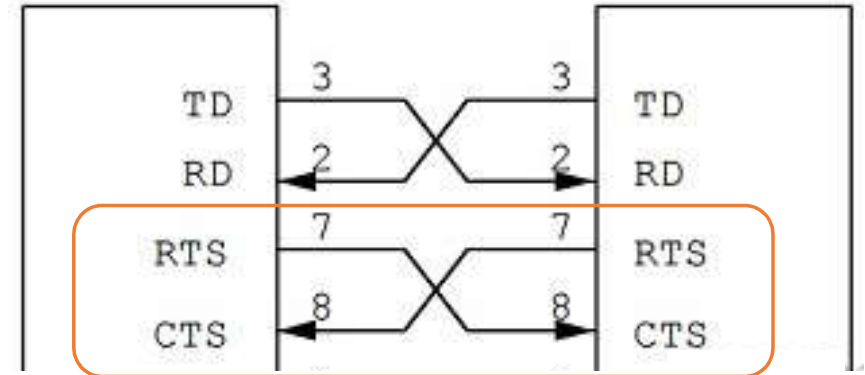
3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

CHƯƠNG 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

3.1 UART

3.1.6 RS232 Protocol

→ with **Hardware Handshaking**, the data from the transmitter is never **lost** or **overwritten**.



- + When the transmitter (DTE) wants to send data: RTS (Ready to Send) = '1'.
- + Then the transmitter waits for CTS (Clear to Send) = '1' ? , hence it keeps on monitoring it. → If the CTS = '0', it means that the receiver (DCE) is **busy** and **not yet ready to receive** data.
- + When the receiver is ready, the CTS = '1' → the transmitter then transmits the data. → **This method is also called as RTS/CTS Handshaking.**

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

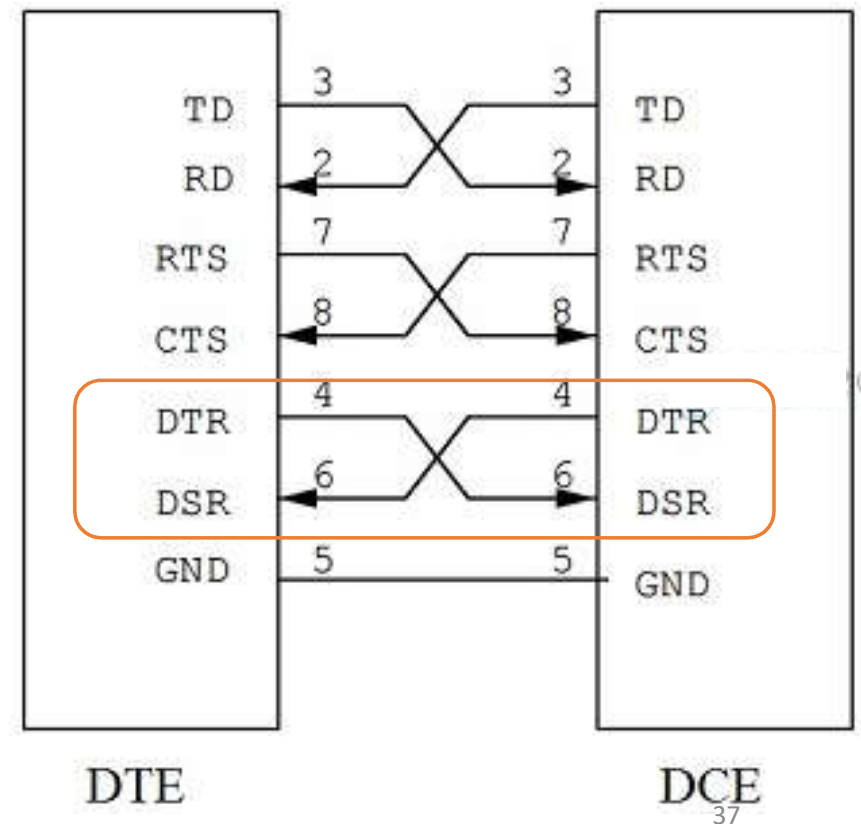
3.1 UART

3.1.6 RS232 Protocol

❑ Hardware Handshaking

+ Additionally, there are two other wires used in Handshaking. They are **DTR** (Data Terminal Ready) and **DSR** (Data Set Ready).

→ These two signals are used by the **DTE** and **DCE** to indicate their individual status. Often, these two signals are used in **modem communication**.



3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

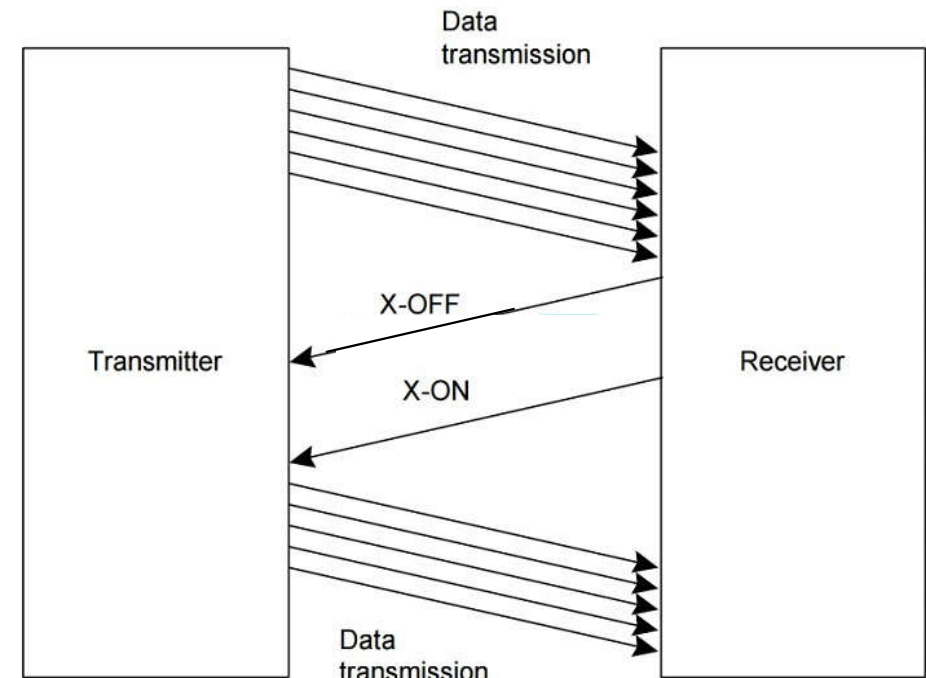
3.1 UART

3.1.6 RS232 Protocol

❑ Software Handshaking

+ Software Handshaking in RS232 involves two special characters for **starting** and **stopping** the communication. These characters are **X-ON** and **X-OFF**.

+ When the receiver **sends an X-OFF signal**, the transmitter **Stops sending** the data. The transmitter **Starts sending** data only after it **receives the X-ON signal**.



3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.1 UART

3.1.6 RS232 Protocol

Limitation of RS232

- + RS232 Protocol requires a **common ground** between the transmitter (DTE) and receiver (DCE) → the reason for **shorter cables**.
- + The signal in the line is **highly susceptible** to **noise**. The noise can be either internal or external.
- + If there is an **increase in baud rate** and **length of the cable**, there is a chance of **cross talk (noise)** introduced by the **capacitance** between the cables.
- + The voltage levels in RS232 are **not compatible** with modern TTL or CMOS logics. We need an external level converter

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

☐ UART

☐ I2C

☐ CAN

☐ SPI

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.2 I2C

3.2.1 Introduction

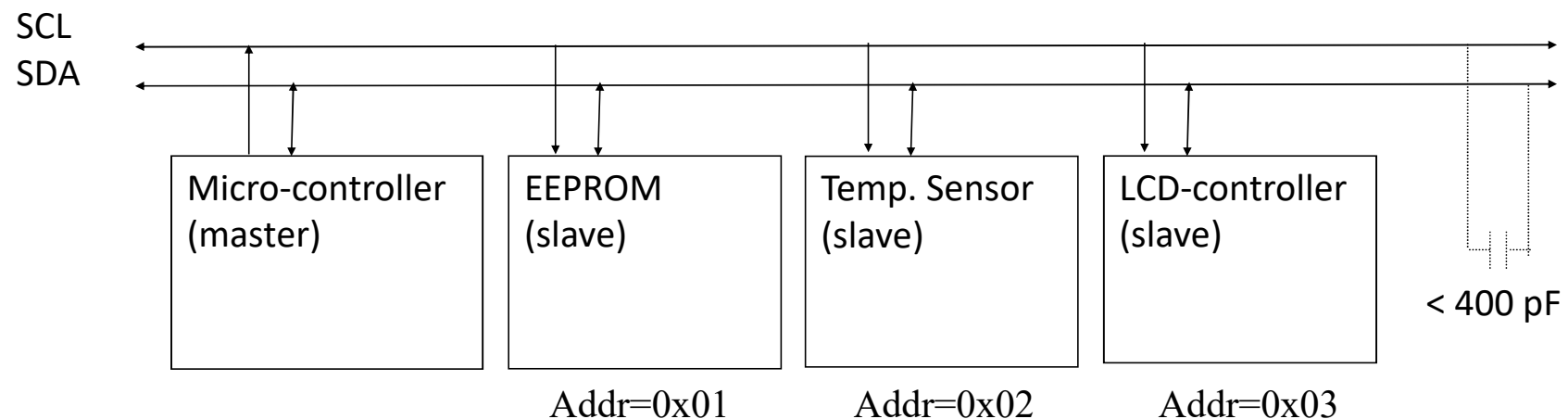
- + The **I2C** (Inter-Integrated Circuit) bus was designed by Philips in the early '80s to allow easy communication between components which reside on the same circuit board.
- + I2C is not only used on single boards but also to connect components which are linked via cable. **Simplicity** and **flexibility** are key characteristics that make this bus attractive to many applications.
- + **Speed**: the original communication speed was defined with a maximum of 100 kbit per second. Besides, there is a **400 kbit fast mode** and a high speed **3.4 Mbit** option available.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.2 I2C

3.2.1 Introduction

+ **The great strength** of this protocol is that it only requires **two wires**, yet can have many connected devices and all of these can transmit and receive data at **high speed**.



3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.2 I2C

3.2.2 Operation

- + The I2C protocol has an **acknowledgement feature** (**ACK** bit) that means a sending device knows that a receiver has accepted the data → I2C is **more robust** in a **noisy environment**.
- + **I2C Device Addressing**: All I2C **addresses** are either 7 bits or 10 bits. The use of 10 bit addresses is rare, and **common** is **7 bit** addresses (*128 devices*).
- + When sending out the 7 bit address, → still **always send 8 bits**. The extra bit (Read/Write bit) is used **to inform the slave**: If the bit is **‘0’** the master is **writing** to the slave, is **‘1’** the master is **reading** from the slave.
 - This Read/Write (**R/W**) bit is **LSB bit** (*Least Significant Bit*).

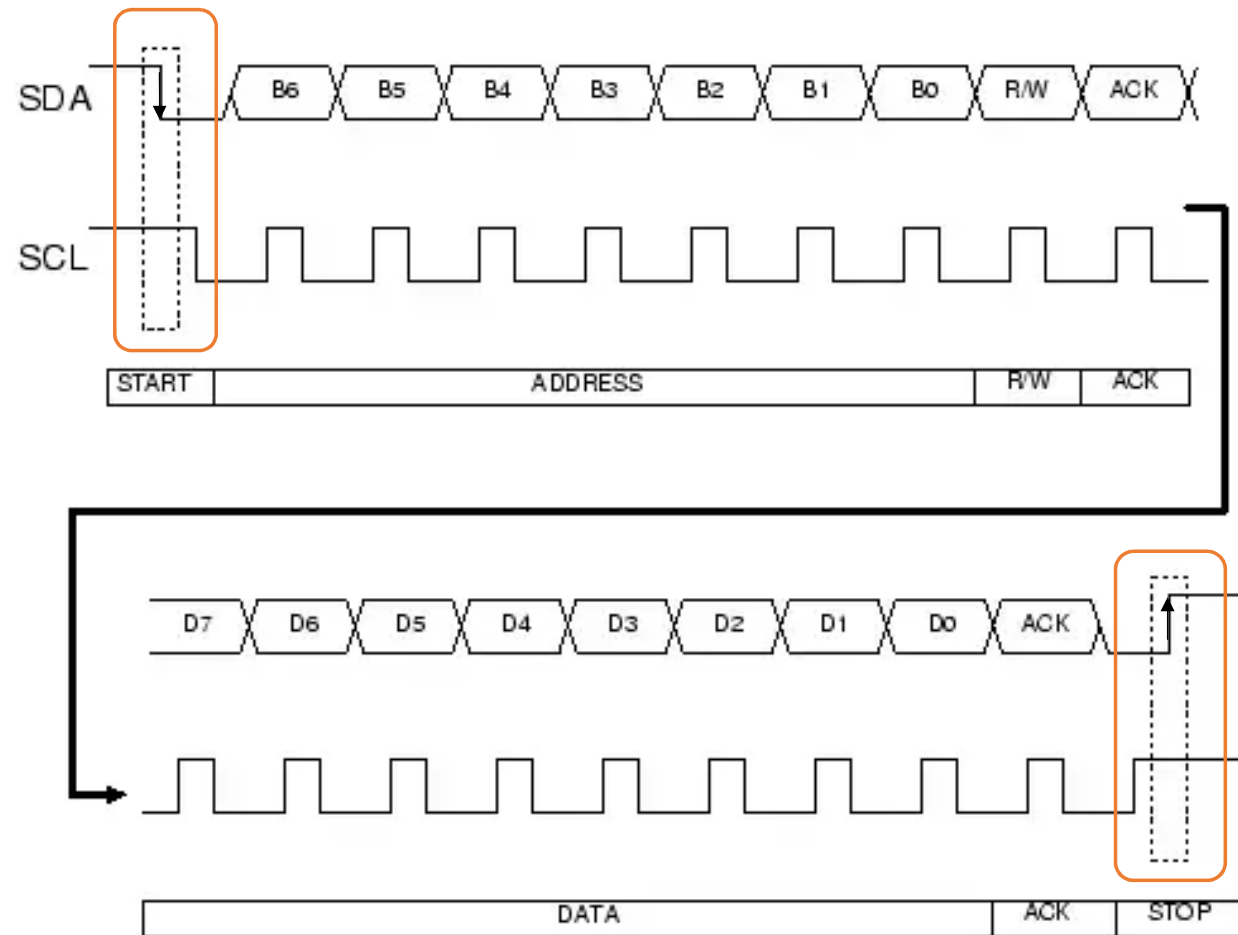
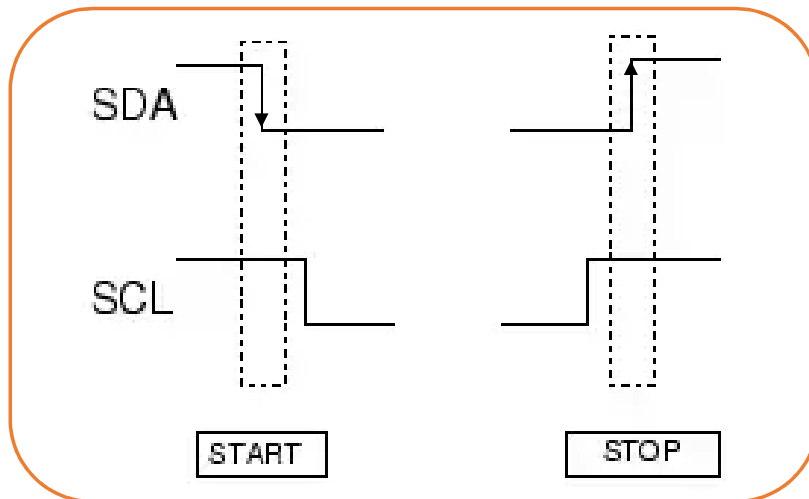
3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

CHƯƠNG 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

3.2 I2C

3.2.2 Operation

- + For the master writing to a slave, the **ACK bit** is generated by the slave.
- + For the master receiving data from a slave, the master generates the **ACK bit**.



3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.2 I2C

Data transfer sequence

A basic Master to Slave read or write sequence for I2C follows steps:

- Send the START bit (**S**).
- Send the slave address (**ADDR**). Usually 7 bits.
- Send the Read(**R**)-'1' / Write(**W**)-'0' bit.
- Wait for/Send an acknowledge bit (**A**).
- Send/Receive the data byte (8 bits) (**DATA**)
- Expect/Send acknowledge bit (**A**).
- Send the STOP bit (**P**).

+ For the master **writing** to a slave, the **ACK bit** is generated by the **slave**.

+ For the master **receiving** data from a slave, the **master** generates the **ACK bit**.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.2 I2C

1) **Data Transfer from master to slave** (The slave generates the ACK signal)

+ A master device sends the sequence: **START bit**, **address W** and then **waits for** an acknowledge bit (**A**) from the slave (*The slave will only generate if its internal address matches the value sent by the master*).

+ If this happens then the **master sends DATA** and **waits for** acknowledge bit (**A**) from the **slave**. The master completes the byte transfer by **generating a STOP bit (P)** (or repeated START).

Instruction sequence data from **master to slave**

START	ADDRESS	W	ACK	DATA	ACK	DATA	ACK	P
-------	---------	---	-----	------	-----	------	-----	---



sent by master



sent by slave

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN



CHƯƠNG 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

3.2 I2C

2) Data Transfer from slave to master (The master generates the ACK signal)

+ A similar process happens when a master reads from the slave but in this case, (*instead of address W*), **R** is sent. After the **data** is transmitted from the slave to the master, the **master** sends the acknowledge (**A**).

+ **Not-Acknowledge (NACK)**. If instead the master does **not want any more data** it must send a **Not-acknowledge** to the slave that it should release the bus. This lets the master send the STOP or repeated START signal.

Instruction sequence data from **slave to master**

START	ADDRESS	R	ACK	DATA	ACK	DATA	NACK	P
-------	---------	---	-----	------	-----	------	------	---



sent by master



sent by slave

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.2 I2C

Not-acknowledge (NACK) bit

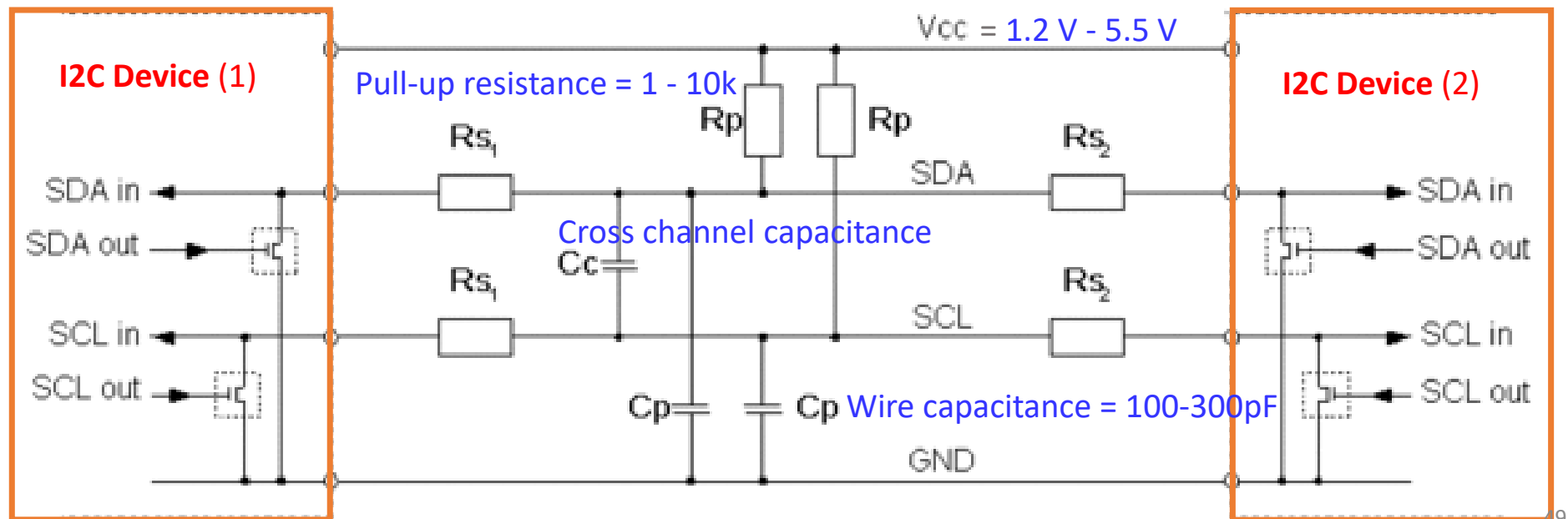
- + **The slave generates the ACK signal:** When a Not-acknowledge is received by the bus master, the transfer has failed and the master must generate a STOP or repeated START to abort the sequence.
- + **The master generates the ACK signal:** Normally the master will generate an acknowledge after it has received data, but to indicate to the slave that no more data is required on the last byte transfer, the master must generate a 'Not-acknowledge'.
 - This indicates to the slave that it should stop sending data. The master can then generate the STOP bit (or repeated START).

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.2 I2C

3.2.2 Operation

Schematic for I2C connection



3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

☐ UART

☐ I2C

☐ CAN

☐ SPI

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN



CHƯƠNG 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

3.3 CAN

3.3.1 Introduction

+ **Controller Area Network (CAN)** is a very reliable, multi-drop asynchronous serial network technology. It was originally developed by Bosch, starting in 1983.

+ **CAN protocol:**

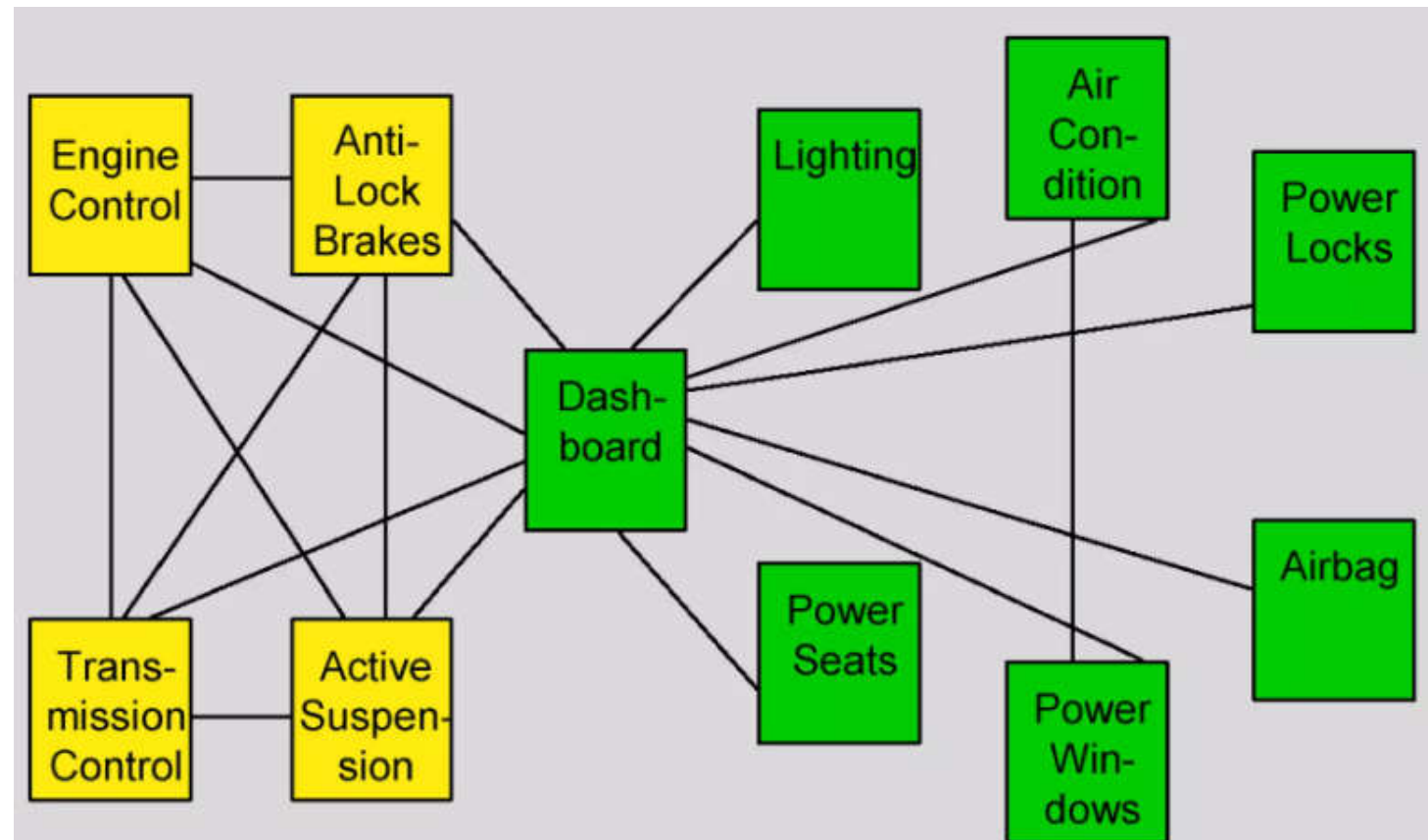
- Robust in noisy environments
- Compact and fast
- It is a message-based protocol

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.3 CAN

3.3.1 Introduction

+ **Before CAN**



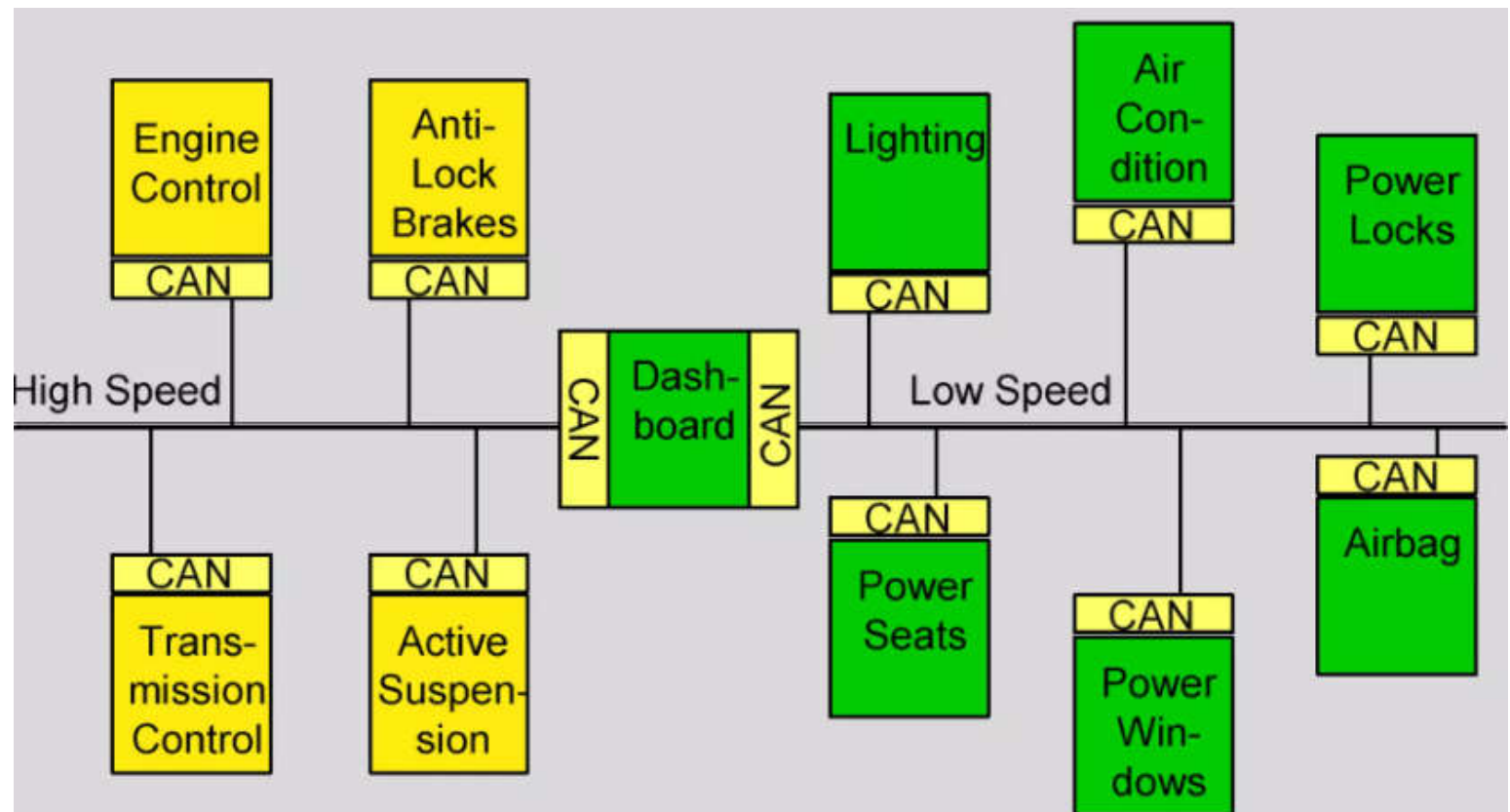
3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

CHƯƠNG 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

3.3 CAN

3.3.1 Introduction

+ After CAN

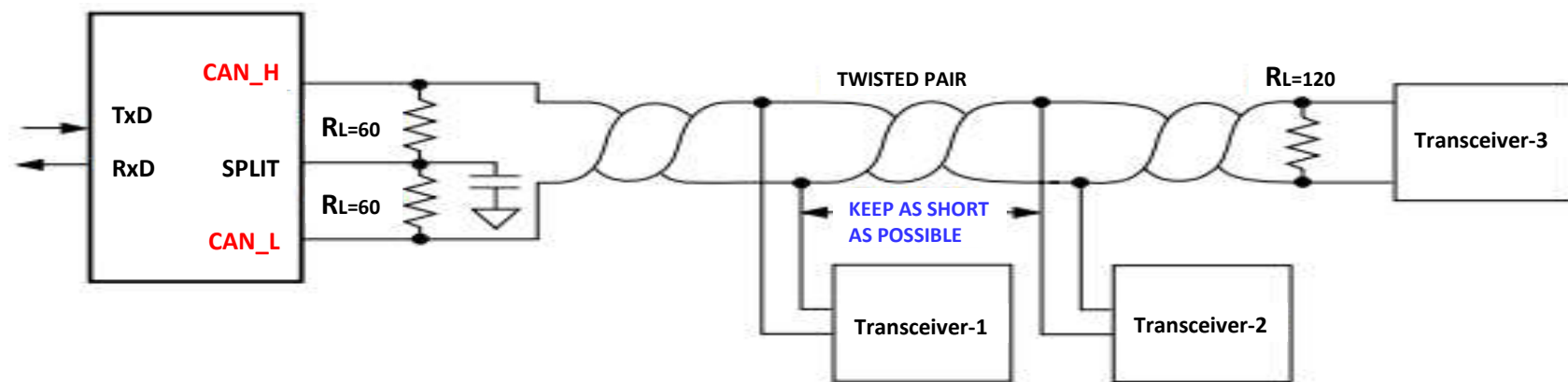


3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.3 CAN

3.3.1 Introduction

+ The **typical medium** is a twisted pair of wires and the maximum network length depends on the network speed chosen (longer distances are achievable at lower speeds), for example: **40m** at **1 Mbps**; **500m** at **125 kbps**; Up to **10km** at **5 kbps**.

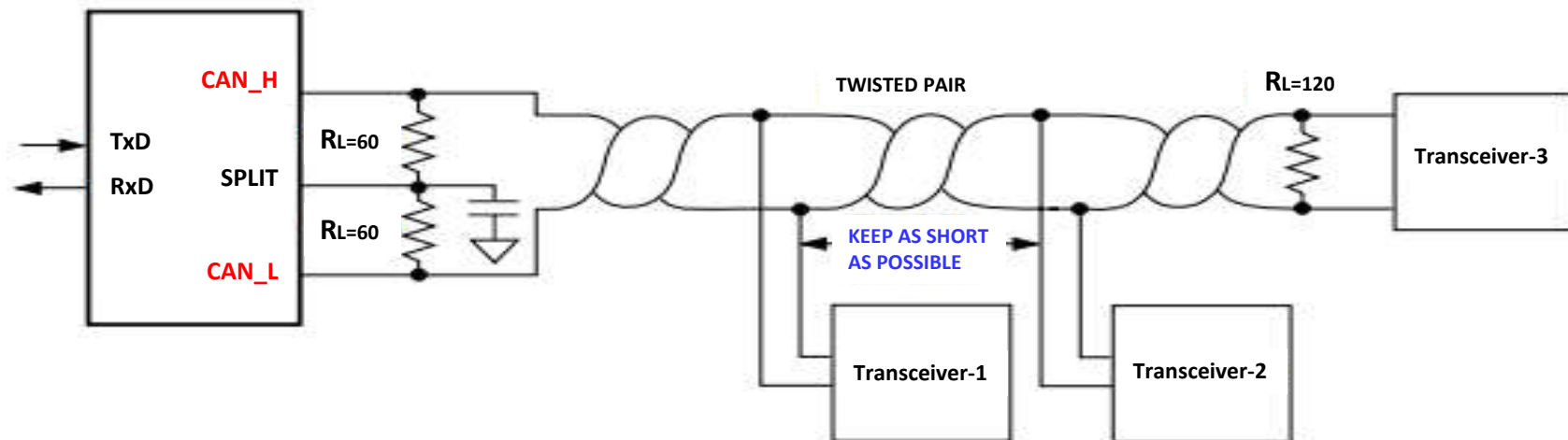


3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.3 CAN

3.3.1 Introduction

+ In a CAN bus all the communicating **devices** are connected to the same two wires, labeled **CAN-High** and **CAN-Low**. At each end, the two wires are connected with a Termination Resistor (120 ohm).



3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

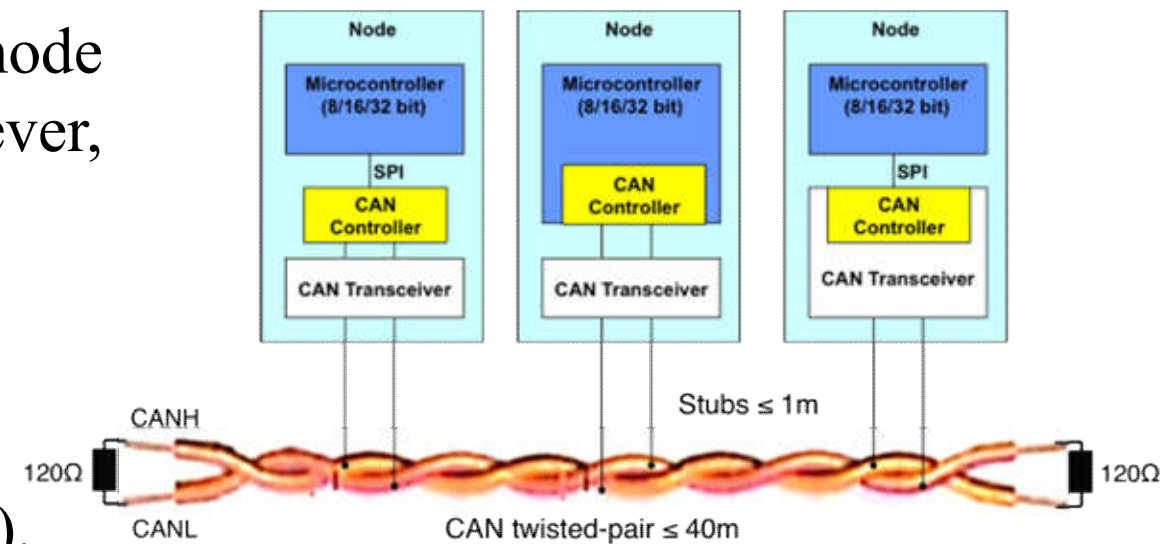
CHƯƠNG 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

3.3 CAN

3.3.1 Introduction

+ There are a variety of CAN node **configuration** possibilities, however, all nodes contain:

- A CAN transceiver,
- A CAN controller,
- An application controller (MCU).



3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.3 CAN

3.3.1 Introduction

- **Application controller (MCU)** is a host processor decides what received messages mean and which messages it wants to transmit itself.
- **CAN controller**
 - **Receiving:** the CAN controller stores received bits serially from the bus until an entire message is available.
 - **Sending:** the host processor stores its transmit messages to a CAN controller, which transmits the bits serially onto the bus.
- **CAN transceiver**

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN



CHƯƠNG 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

3.3 CAN

3.3.1 Introduction

- **CAN transceiver**
 - **Receiving:** it adapts signal levels from the bus to levels that the CAN controller expects and has protective circuitry that protects the CAN controller.
 - **Transmitting:** it converts the transmit-bit signal received from the CAN controller into a signal that is sent onto the bus.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.3 CAN

3.3.2 Message Format

CAN Message Types

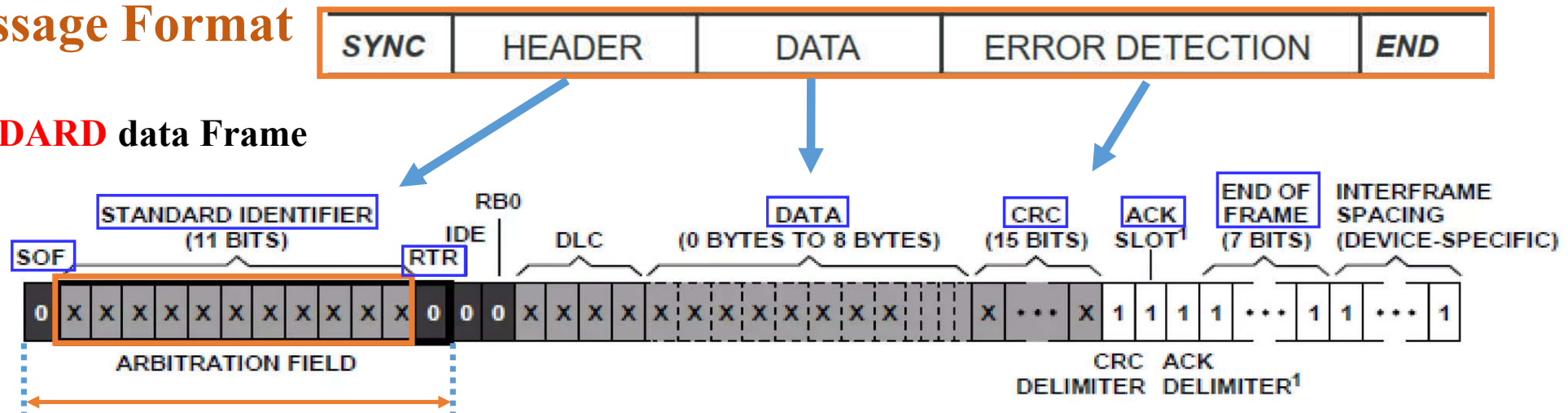
- **Data Frame:** used to transmit data.
- **Remote Frame :** used to request data transmission.
- **Error Frame:** sent by a node that detects an error.
- **Overload Frame:** sent by a node to request a delay in transmission.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

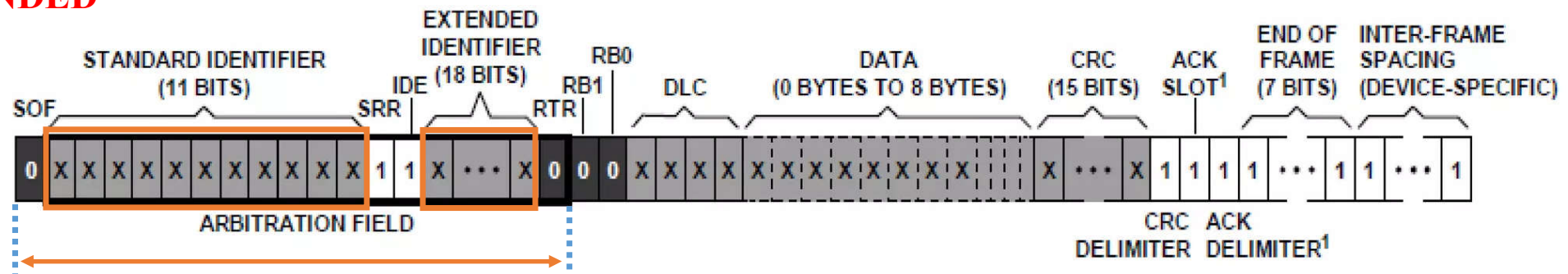
3.3 CAN

3.3.2 Message Format

STANDARD data Frame



EXTENDED

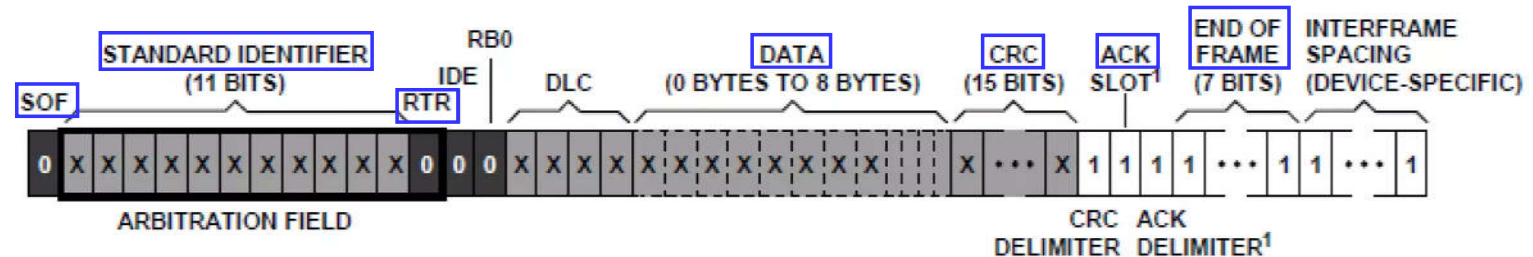


3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

CHƯƠNG 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

3.3 CAN

3.3.2 Message Format

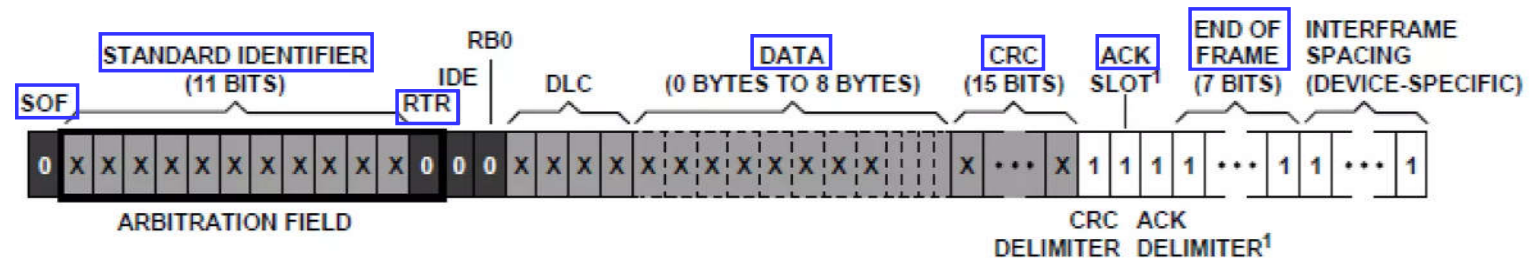


- **Start of frame (SOF)**: Single dominant bit (=‘0’).
- **Message Identifier**: The Lower the value the Higher the priority of the message. Its length is either 11 (automobile) or 29 bits long(aerospace)
- **Remote Transmission Request (RTR)** seldom used.
- **Identifier Extension (IDE)** distinguishes **standard(0)/extended** CAN.
- **Data Length Code(DLC)** contains number of data bytes be transmitted
- **Control Field (CONTROL)** **specifies** the number of bytes of data to follow (0-8).

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.3 CAN

3.3.2 Message Format



- **Data Field (DATA):** length 0 to 8 bytes.
- **CRC field:** containing a fifteen bit **Cyclic Redundancy Check** code.
- **Acknowledge field (ACK):** to confirm data has been received.
 - An empty slot which will be filled by any and every node that receives the frame.
 - It does mean that **at least one node** on the whole network **got data**.
- **End of Frame (EOF).**

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.3 CAN

3.3.3 CAN Arbitration

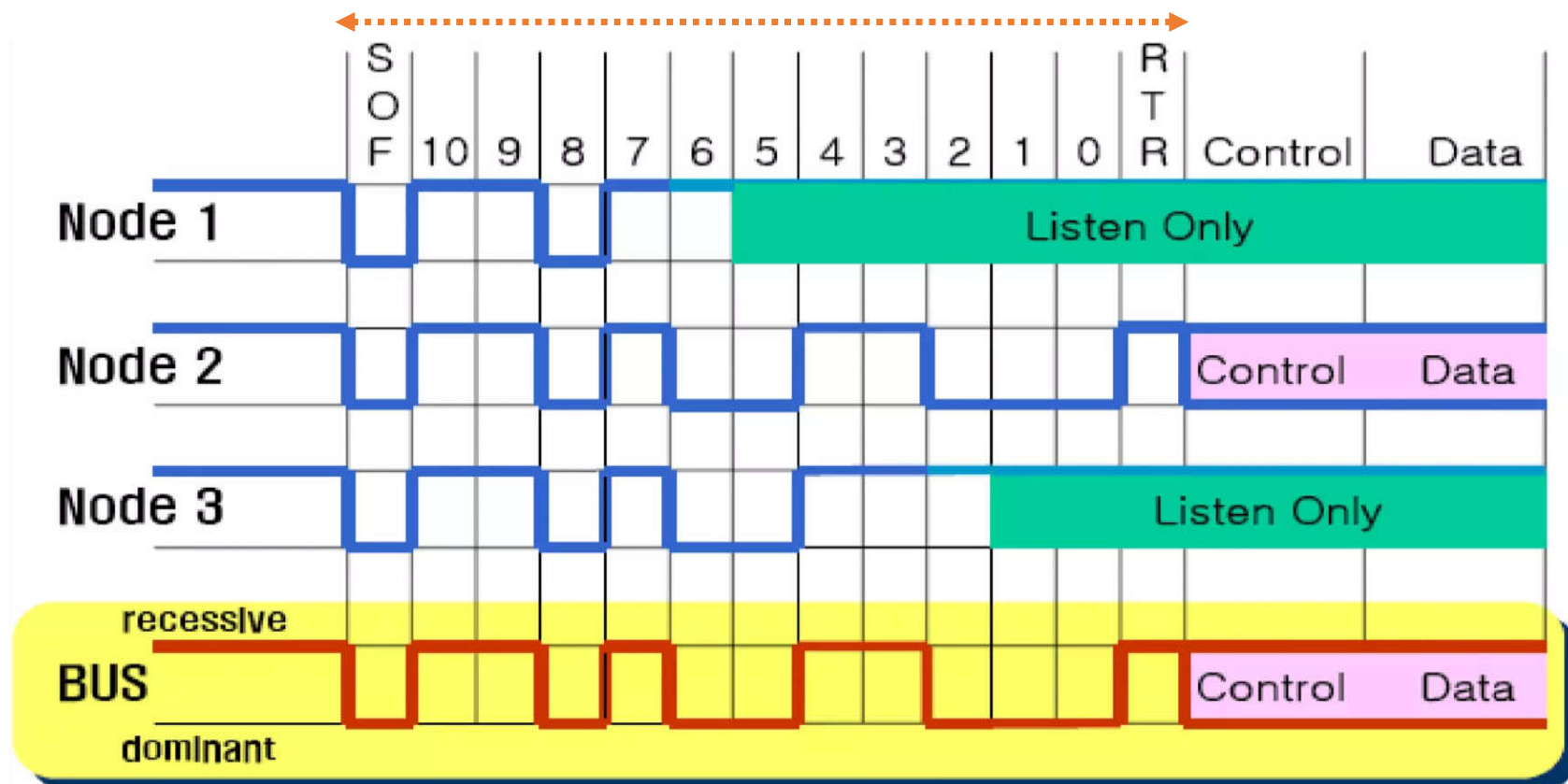
- CSMA/CA (Carrier Sense Multiple Access/Collision Avoidance).
- All nodes must **wait** for an idle bus condition.
- If **two nodes** begin transmitting **simultaneously**, they then participate in an arbitration process.
- Wire-AND mechanism.
- The node with the **lower ID** number **wins** the arbitration and continues transmitting its message.
- The **loser** of the arbitration backs off and re-tries.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.3 CAN

3.3.3 CAN Arbitration

CSMA/CA & Wired-AND Logic



INDUSTRIAL
UNIVERSITY
OF HOCHIMINH CITY

3.3 CAN

- When several messages access the bus **at the same time**, the one with the **higher priority** win.
- The identifier with the *lowest binary number* has the *highest priority*.
- Access conflicts on the bus are resolved by a “**wire-AND**” mechanism.

The diagram shows the timing of a successful CSMA/CD transmission. The horizontal axis represents time, divided into slots for signaling (S, O, F, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0, R, T, R) and data. The vertical axis shows the state of three nodes and the bus.

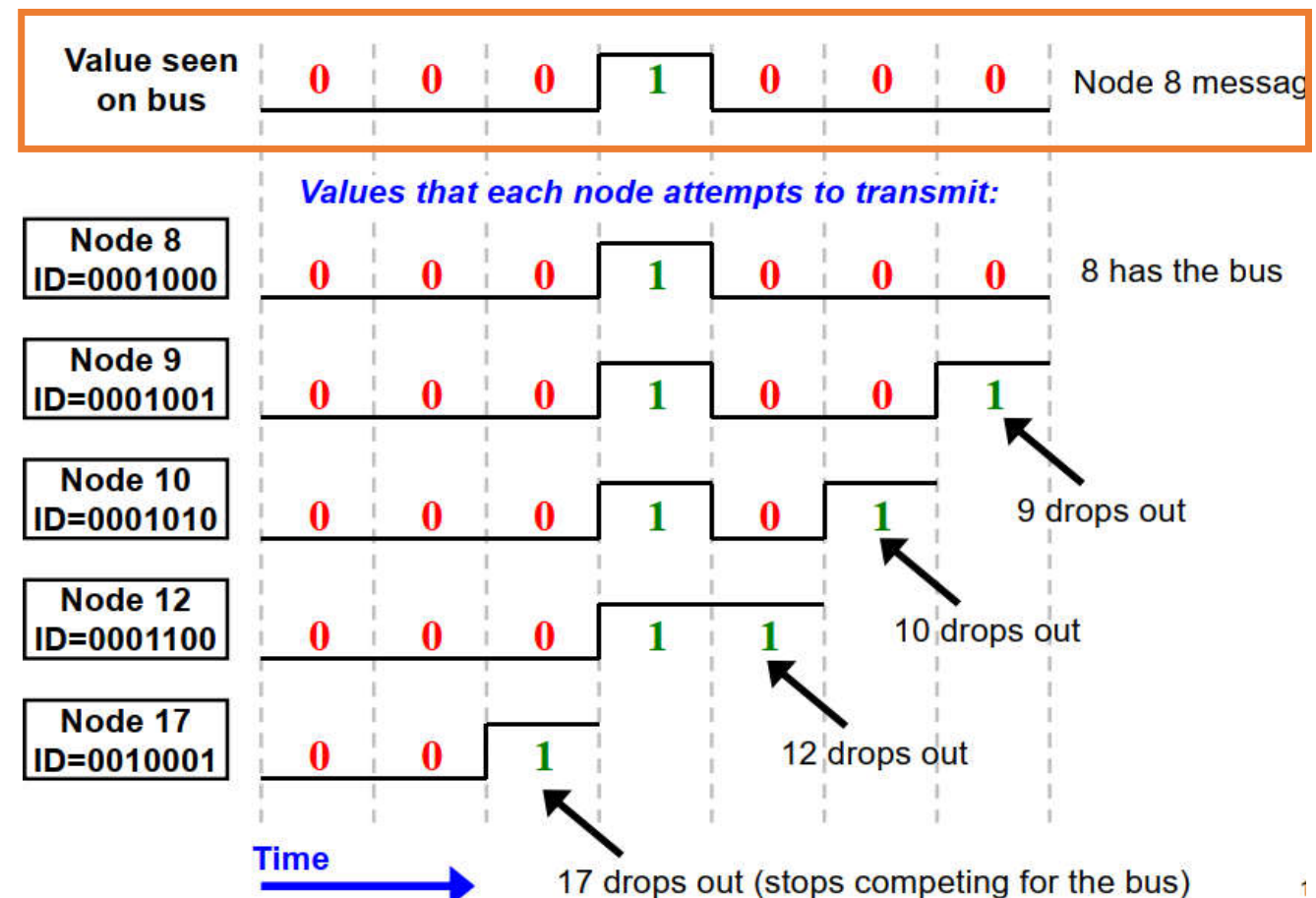
- Node 1:** Transmits during the 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0 slots. It then enters a "Listen Only" state.
- Node 2:** Receives during the 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0 slots. It then transmits during the R, T, R slots. It then enters a "Control Data" state.
- Node 3:** Listens during the 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0 slots. It then enters a "Listen Only" state.
- BUS:** The bus signal is recessive (high) during the 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0 slots. It becomes dominant (low) during the R, T, R slots. It then enters a "Control Data" state.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.3 CAN

3.3.3 CAN Arbitration

CSMA/CA & Wired-AND Logic



3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.3 CAN

3.3.4 Non-Return to Zero Encoding (NRZ) methods

Definition. NRZ encoding is a **binary encoding scheme**.

- There is **no return to zero voltage** between bits.
- The signal only changes when the bit value changes (**0** → **1** or **1** → **0**)

Example (*UART Data Frame*): **Start** (0) → Data (**NRZ-encoded**) → **Stop** (1).

Important

- **Simple** and **easy** to implement in hardware.
- **Maximizes data transmission efficiency** with fewer transitions.
- **Reduces power consumption** in embedded & networking systems. ⁶⁷

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.3 CAN

3.3.4 Non-Return to Zero Encoding (NRZ)

- **Send 0/1 logic**. Simplest solution is to limit data length to perhaps **8 bits**.
- SYNC ('0') and END ('1') are opposite values, guaranteeing two edges per message.
 - This is the technique commonly used on computer serial Ports / UARTs.

NRZ Encoding Example: 1101 0001



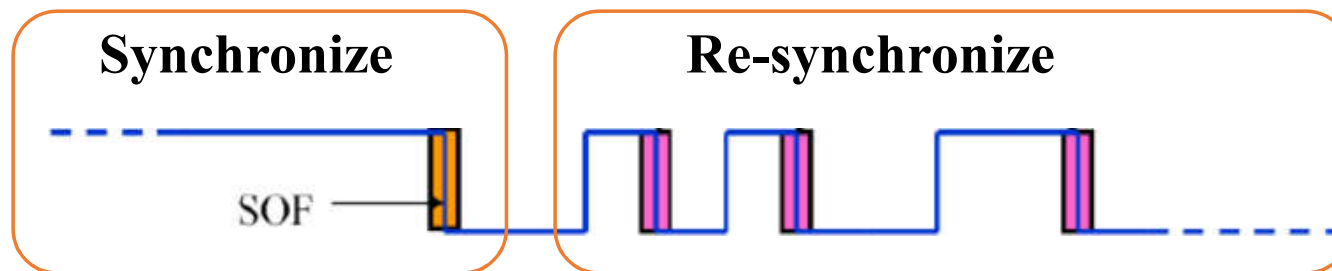
3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.3 CAN

3.3.4 Non-Return to Zero Encoding (NRZ)

→ Long NRZ messages cause problems

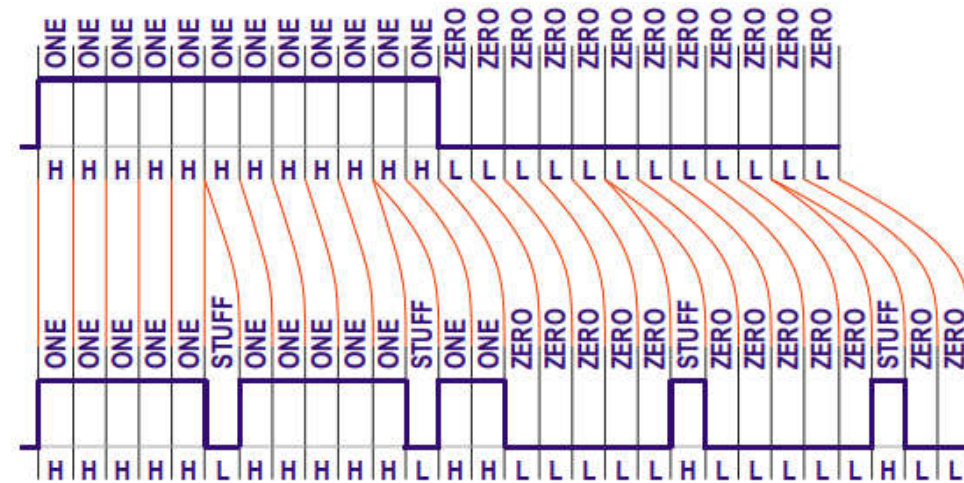
- Clock drift means that if there are no edges (value changes from 0 to 1 or 1 to 0), receivers lose track of bits.
- Periodic edges allow receiver to **Re-synchronize** to sender clock → **Re-synchronize** or **continuous synchronization** in turn enables the **Receiver** to be able to properly read the messages.



3.3 CAN

- Typical: add an opposite-valued stuff bit after every 5 identical bits.

→ At the **receiver end**, **the stuffed bit** is **removed** to recover the original data.



3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.3 CAN

3.3.5 CAN Data Protection – Error Detection

- If one or more **errors are detected**, the transmission is aborted.
→ This prevents all other nodes from accepting the message.
- Re-transmission is automatic. If **errors continue**, then the **node** may **switch itself off** → to prevent the bus from being tied up.
- Error detection is done on two levels: **Message** and **bit** level.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN



CHƯƠNG 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

3.3 CAN

3.3.5 CAN Data Protection – Error Detection

❑ Message level

- CRC= Cyclic Redundant Check sum
- Frame Check = compares message to **fixed** format and frame size.
- ACK errors= if transmitter does **not receive** an **ACK** signal from the receives.

❑ Bit level

- Monitoring= The transmitter monitors **the bus signal** as it sends the message and compares the bit sent to the bit received.
- Bit **Stuffing**= After **five consecutive equal bits**, the transmitter **inserts a stuff bit** with a compliment value into the bit stream. The receivers remove this stuff bit. 72

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.3 CAN

3.3.6 CAN Advantages – Disadvantages

❑ Advantages

- High performance under light loads / Low cost / Reliable / Robust.

❑ Disadvantages

- Unfair access: Node with high priority can hog the network.
- Starvation for some particular nodes.

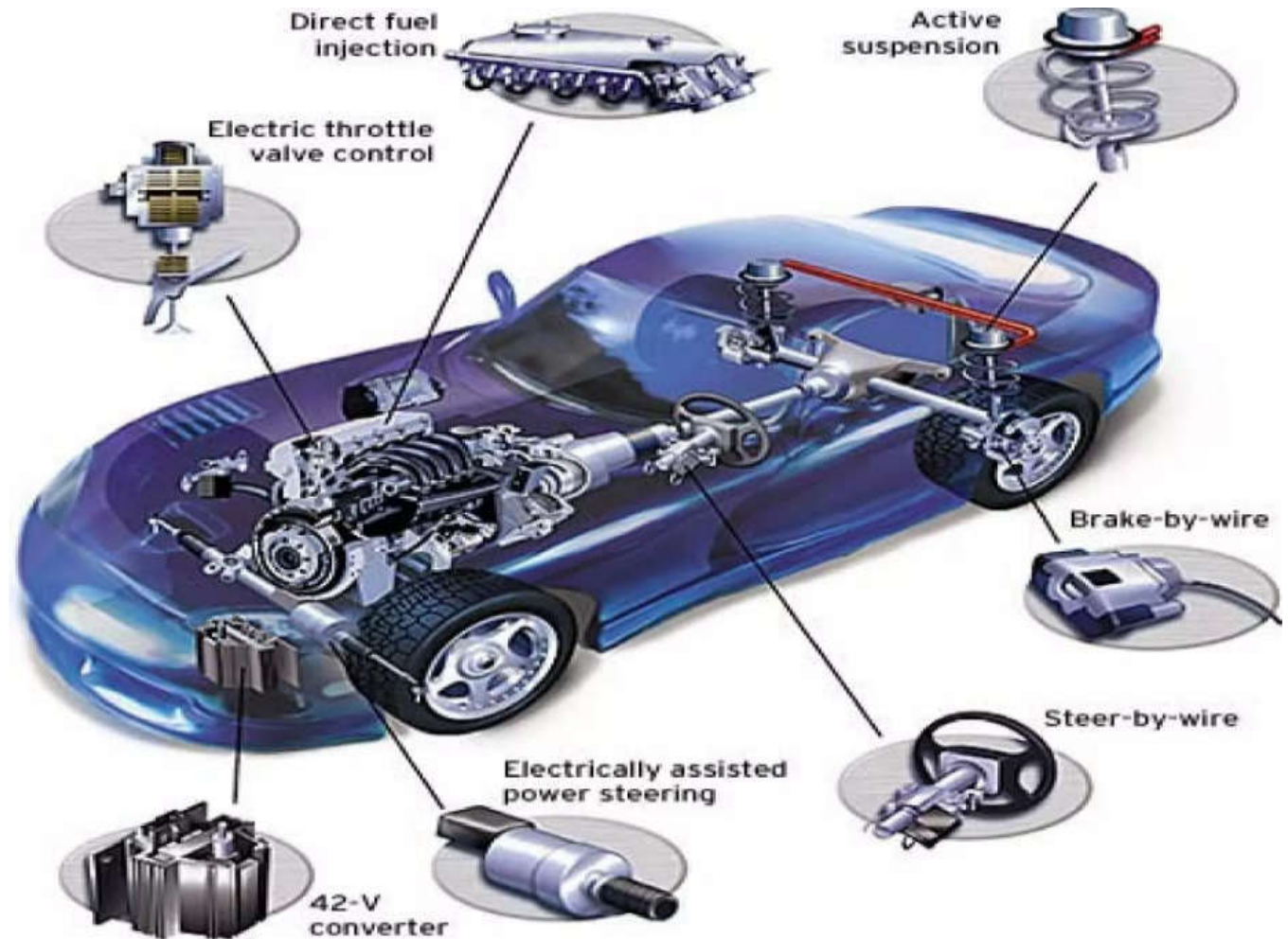
3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

CHƯƠNG 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

3.3 CAN

3.3.7 CAN Applications

- Automotive



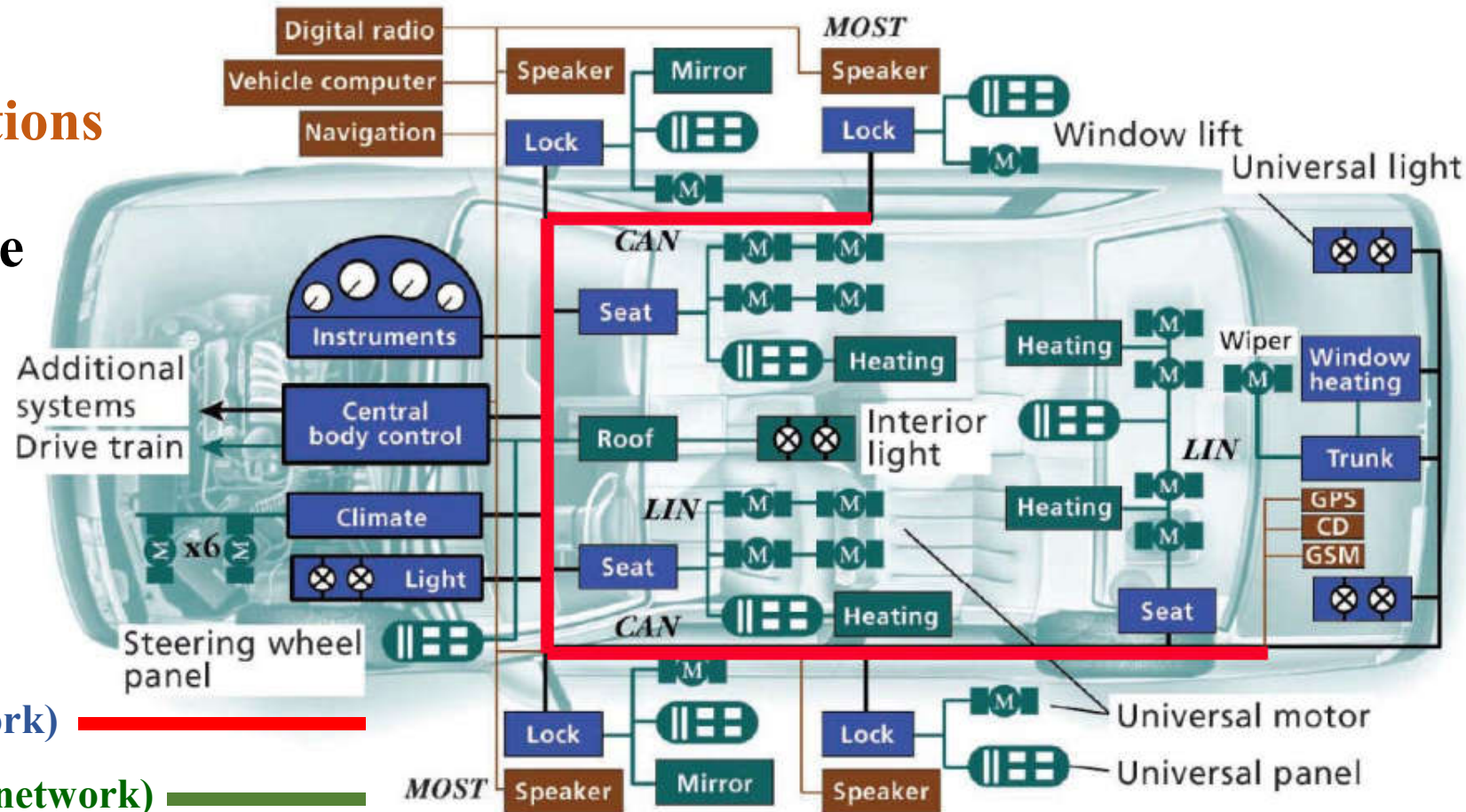
3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

CHƯƠNG 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

3.3 CAN

3.3.7 CAN Applications

- Automotive



CAN (Controller area network) —

LIN (Local interconnection network) —

MOST (Media-oriented systems transport) —

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.3 CAN

3.3.7 CAN Applications

- Military vehicles
- Industrial machinery
- Medical systems
- Agricultural machinery
- Marine control and navigation
- Elevator control systems

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

☐ UART

☐ I2C

☐ CAN

☐ SPI

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.4 SPI

3.4.1 Introduction

- + **Serial Peripheral Interface (SPI)** is a simple serial communication protocol using **4 wires** (4-wire bus).
- + Used to communicate across **small distances**.
- + **Multiple Slaves**, Single Master.
- + **Synchronized**.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.4 SPI

3.4.1 Introduction

- + Fast, Easy to use, Simple.
- + Has some **advantages** over **I2C**:
 - SPI can communicate at **much higher data rates** than I2C.
 - Also, when multiple slaves are present, SPI requires **no addressing** to differentiate between these slaves.

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.4 SPI

3.4.2 Capability

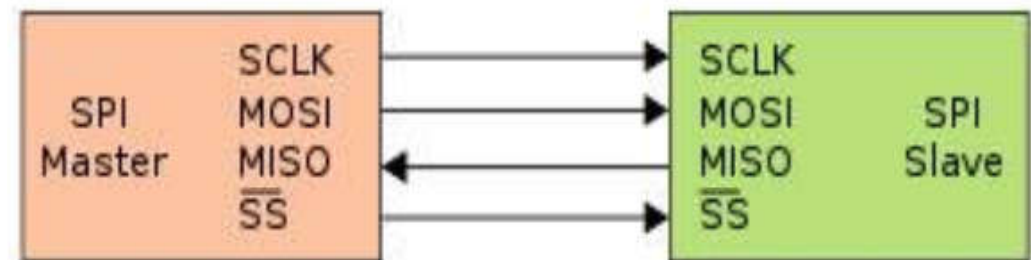
- **Full Duplex** → communicating in two directions **at the same time**.
- There is **no limit** for data transmission **speed** in SPI (Mps).
- SPI has an **8 or 16 bit frame** consisting only data.
- **Multiple slaves** → daisy-chaining possible (a wiring scheme in which multiple devices are wired together **in sequence** or **in a ring**).

3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.4 SPI

3.4.3 Communication method

- SPI runs using a **master/slave set-up** and can run in **full duplex** mode (signals can be transmitted between the master and the slave simultaneously).
- Wires:
 - Master out Slave in (**MOSI**)
 - Master in Slave out (**MISO**)
 - System Clock (**SCLK**)
 - Slave Select (**SS**)

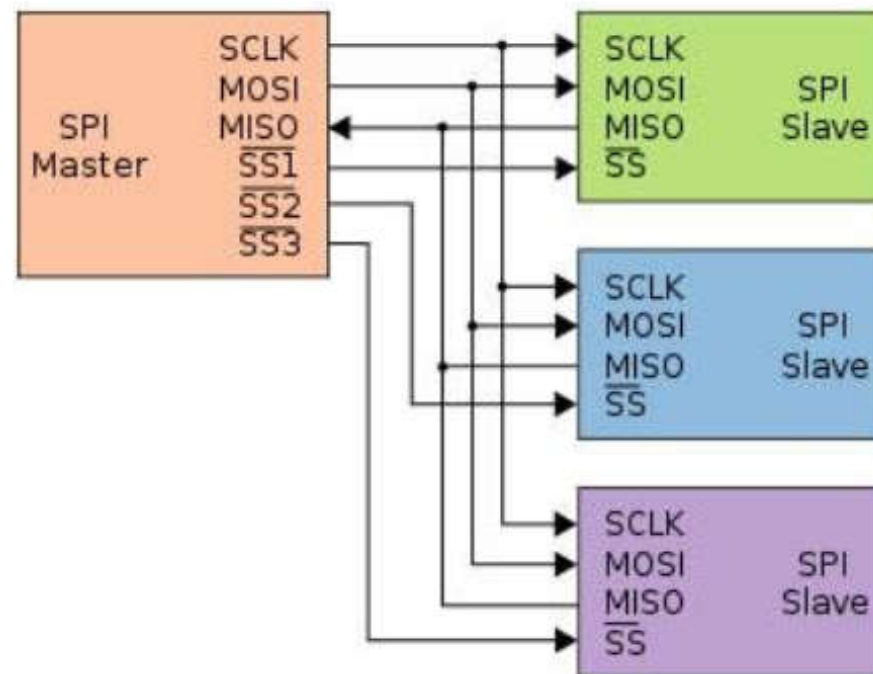


3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.4 SPI

3.4.3 Communication method

- Master and multiple independent slaves.

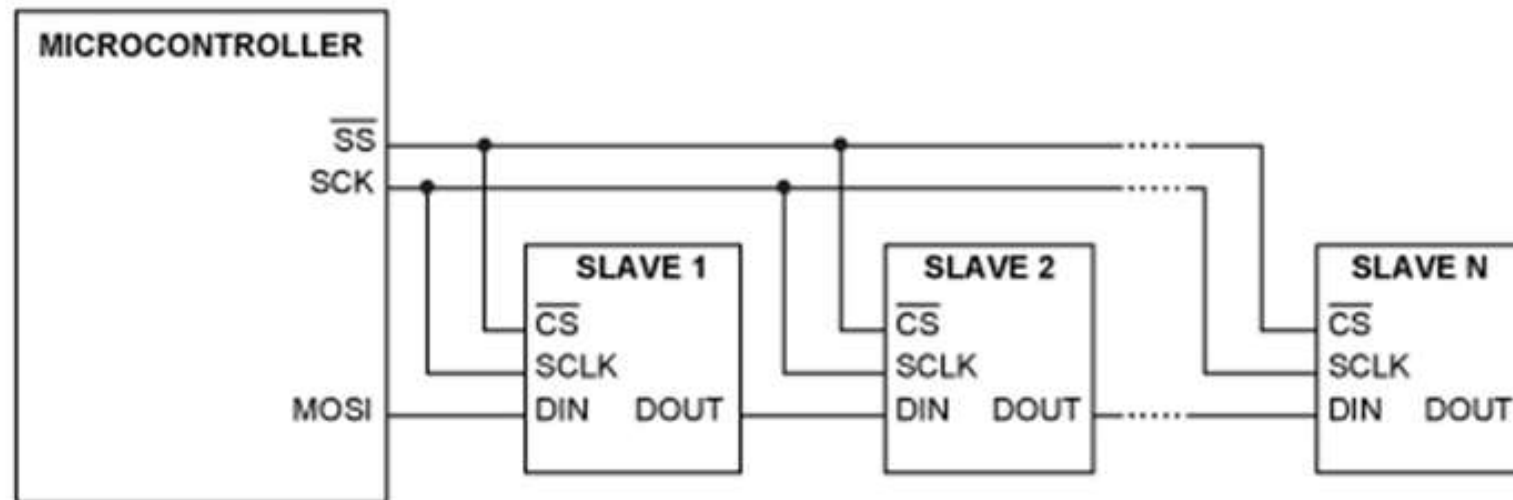


3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.4 SPI

3.4.3 Communication method

- Master and multiple **daisy-chained slaves**.

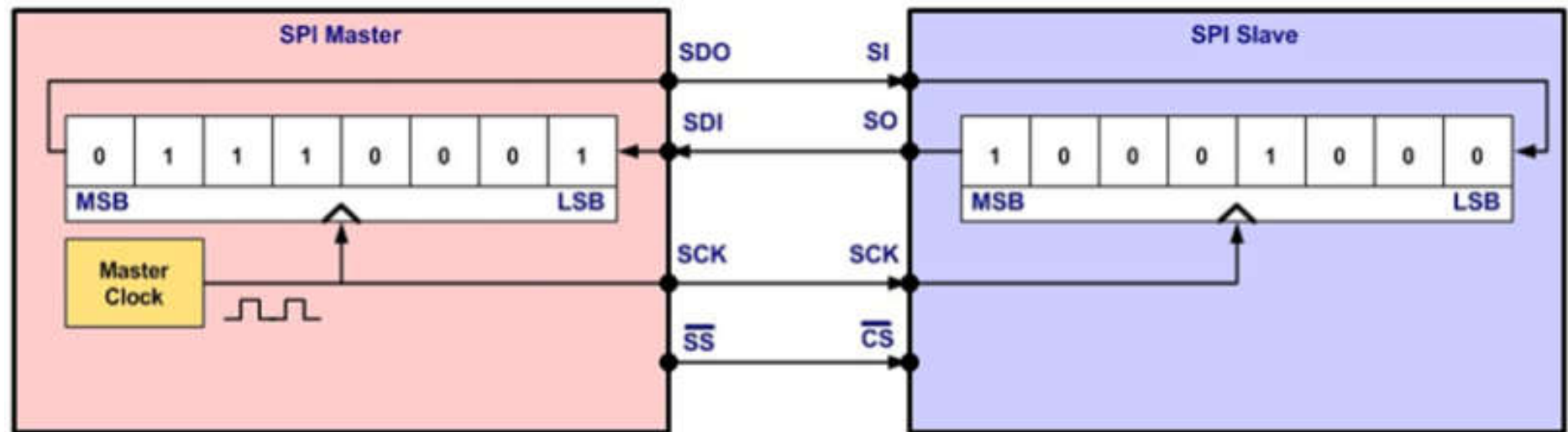


3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.4 SPI

3.4.3 Communication method

- Shifting detail.



3. MỘT SỐ KỸ THUẬT GIAO TIẾP CƠ BẢN

3.4 SPI

3.4.4 Advantages – Disadvantages

❑ Advantages

- Fast and easy.

❑ Disadvantages

- SS makes multiple slaves very complicated.
- No acknowledgement ability.
- No arbitration.
- No flow control.